

GIORNO 08

Leggere e scrivere file: i dati che restano quando il programma si chiude

Cosa saprete fare stasera

Aprire un file di testo con `with open(...)` e leggerlo nel modo giusto:

tutto insieme, a righe, una riga alla volta

Ripulire le righe dall'a capo, dagli spazi e dalle righe vuote

Scrivere un file senza cancellare per sbaglio quello che c'era: `w`, `a`, `x`

Costruire con **pathlib** percorsi che funzionano da qualunque cartella lanciate

Controllare che un file esista prima di aprirlo, e leggere dal basso il traceback

Trasformare un file a righe in una lista o in un dizionario, e salvare il

risultato in un formato che un altro programma sa rileggere

Non si butta via niente: cicli, funzioni, liste e dizionari restano identici.

Cambiano le prime righe, quelle che caricano i dati,

e le ultime, quelle che li salvano.

1. RIPASSO E DOMANDE

Il registro che sparisce, e il primo import del corso

1.1 La lista di tuple del Giorno 07: il registro che sparisce

```
# 1. Il registro del Giorno 07 S14.1: una LISTA DI TUPLE scritta a mano nel

#     codice. Una tupla per candidatura, quattro campi in ordine fisso.

candidature = [

    ("Rossi Anna", 3, 78, "IDONEO"),

    ("Bianchi Marco", 1, 55, "NON IDONEO"),

    ("Verdi Luca", 6, 91, "IDONEO"),

]

# 2. Unpacking nel for: un nome per colonna. L'accumulatore parte da zero.

somma_test = 0

for nome, anni, punteggio, esito in candidature:

    somma_test += punteggio

    print(f"{nome:<15}{punteggio:>4} {esito}")

print(f"Media del test: {somma_test / len(candidature):.2f}")


# 3. Una candidatura arriva a programma avviato, e append la accoda.

candidature.append(("Neri Sara", 2, 84, "IDONEO"))

print("Righe nel registro:", len(candidature))


# TRABOCCHETTO: Neri Sara esiste finche' il programma gira. Rilanciatelo e

#     il registro riparte dalle tre righe scritte sopra: l'append non ha

#     lasciato traccia da nessuna parte, e nessuno ve lo segnala.
```

1.2 Prerequisiti operativi: split, strip, unpacking, for, get

Dal Giorno	Che cosa serve oggi
01	print con sep ed end, il carattere di escape \n, leggere un traceback dal basso
02	metodi di stringa (.strip(), .replace(), .count(), .startswith()), f-string
03	.split(";") e l'unpacking a tre nomi, la virgola italiana, pulire dopo lo split
04	if, in su una stringa, and/or, la validazione preventiva con .isdecimal()
05	contatore, accumulatore, continue, il ciclo con sentinella
06	funzioni che RESTITUISCONO, valori di default, due valori con return
07	lista, dizionario, conta-occorrenze con .get(k, 0) + 1, lista di tuple

Il lavoro nuovo di oggi sta in poche righe: aprire, leggere, scrivere. Tutto il resto lo sapete già, e lo userete dalla prima riga. Se una di queste cose vi sembra sfocata, recuperatela adesso: nel pomeriggio serve in ogni esercizio.

1.2 Sette giornate su una riga sola

```
# La riga come arrivera' dal file: spazi davanti e \n in coda compresi.

riga = "  LS-2026-MI-0042;NORD;12.5\n"

LISTINO = {"NORD": 8.50, "CENTRO": 7.00, "SUD": 9.50, "ISOLE": 14.00}


def costo_zona(zona, listino):

    # .get() con il ripiego (Giorno 07 $11.1): niente KeyError sulla zona

    # sconosciuta, e la funzione RESTITUISCE (Giorno 06 $5.1).

    return listino.get(zona, 0.0)


# 1. .strip() toglie spazi e a capo ai due estremi (Giorno 02 $10.2).

# 2. .split(";") e unpacking: tre pezzi, tre nomi (Giorno 03 $7.2-7.3).

codice, zona, peso_testo = riga.strip().split(";")

peso = float(peso_testo)

prezzo = costo_zona(zona, LISTINO)

print(f"codice:<18){zona:<8){peso:>6.1f} kg{prezzo:>8.2f} euro")


# 3. Contatore dentro un for sui caratteri (Giorno 05 $5.2).

cifre = 0

for carattere in codice:

    if carattere.isdecimal():

        cifre += 1

print("Cifre nel codice:", cifre)


# TRABOCCHETTO: senza .strip() il primo pezzo sarebbe "  LS-2026-MI-0042",

#   con gli spazi attaccati. Nessun errore: un codice che non combacia con

#   nessun altro. float() invece tollererebbe il "12.5\n" (spazi e a capo ai

#   bordi li ignora), ed e' proprio per questo che il difetto non si vede.
```

1.3 Perimetro: il primo import, e che cosa resta fuori

Modulo	A che cosa serve oggi
pathlib	costruire percorsi, controllare che un file esista, creare cartelle
os.path	lo stesso lavoro nel modo vecchio, per riconoscerlo nel codice altrui
json	salvare liste e dizionari così come sono
shutil	fare una copia di sicurezza di un file

Da oggi si importa SOLO dalla libreria standard, cioè dai moduli che arrivano con Python e non si installano a parte, e solo quelli che servono ai file. Un MODULO è un file di Python scritto da altri, con funzioni pronte.

1.3 from pathlib import Path: il primo import del corso

```
# 1. Il PRIMO import del corso. Si legge: "dal modulo pathlib, che sta nella
#   libreria standard, prendi il nome Path". Da qui in poi Path si usa
#   come print o len, senza installare niente.

from pathlib import Path

# 2. Un percorso costruito con la barra: "il file ticket.txt DENTRO la
#   cartella dati". Il capitolo 7 spiega perche' non si incollano stringhe.

percorso = Path("dati") / "ticket.txt"
mancante = Path("dati") / "ordini_ieri.txt"

# 3. La filosofia del Giorno 04 $13.1: prima si controlla, poi si usa. La'
#   era .isdecimal() prima di int(); qui e' .is_file() prima di aprire.

quantita_testo = "12"

if quantita_testo.isdecimal():

    print("Quantità valida:", int(quantita_testo))

if percorso.is_file():

    print("File trovato:", percorso.name)

if not mancante.is_file():

    print("[!] File non trovato:", mancante.name)

# TRABOCCHETTO: si stampa .name, non il percorso intero. Su Windows lo
#   stesso percorso esce con le barre rovesciate, e uno stesso programma
#   stamperebbe due cose diverse su due computer (capitolo 7.5).
```

1.3 Il confine della giornata: si previene con un if

Oggi incontrerete errori nuovi: il file che non c'è, la codifica sbagliata, la cartella che manca.

La risposta di oggi è quella già affrontata:

si PREVIENE con un if, controllando prima di agire.

Quando un errore non si può prevenire, lo si provoca apposta, si legge il traceback e si capisce che cosa è successo.

Esiste uno strumento che raccoglie l'errore dopo che è successo:

ci arriveremo quando arriveremo alle eccezioni.

Restano fuori: librerie da installare, fogli Excel, cancellazione di file e cartelle, moduli di registrazione automatica degli eventi.

2. CHE COS'È UN FILE DI TESTO

Caratteri in fila, qualche `\n`, e una tabella per le accentate

2.1 Memoria e disco: la variabile sparisce, il file resta

```
def turno_di_mattina():  
    # Registra tre accessi: la lista e' LOCALE e muore con la funzione.  
    accessi = ["Serra;anagrafe", "Rinaldi;tributi", "Motta;anagrafe"]  
  
    # 1. Le stesse righe copiate su disco. I dettagli di open() sono il  
    # capitolo 3: qui si guarda solo l'effetto.  
    with open("registro.txt", "w", encoding="utf-8") as f:  
        for accesso in accessi:  
            f.write(accesso + "\n")  
  
    print("Accessi registrati in memoria:", len(accessi))  
  
def il_giorno_dopo():  
    # 2. Nessuna lista, nessun parametro: i dati arrivano SOLO dal disco.  
    with open("registro.txt", "r", encoding="utf-8") as f:  
        contenuto = f.read()  
        print(contenuto, end="")  
  
turno_di_mattina()  
print("--- il giorno dopo ---")  
il_giorno_dopo()  
  
# TRABOCCHETTO: dentro il_giorno_dopo() la variabile accessi non esiste.  
# Scrivere print(len(accessi)) li' si ferma con NameError: name 'accessi'  
# is not defined. E' la variabile locale del Giorno 06 $11.2: finita la  
# funzione, sparisce. Il file invece e' ancora sul disco.
```

2.1 L'output, e il file che resta nella cartella

```
Accessi registrati in memoria: 3
--- il giorno dopo ---
Serra;anagrafe
Rinaldi;tributi
Motta;anagrafe
```

registro.txt è comparso nella cartella da cui avete lanciato lo script: apritelo con il blocco note, sono tre righe di testo. Analogia: la memoria è la lavagna della sala riunioni, che a fine giornata qualcuno cancella; il disco è il quaderno protocollo nel cassetto.

2.2 Un file di testo è una stringa lunga con dentro i \n

```
with open("tre_codici.txt", "w", encoding="utf-8") as f:
    f.write("LS123455\nLS400004\nLS314154\n")

with open("tre_codici.txt", "r", encoding="utf-8") as f:
    testo = f.read()

# 1. read() consegna TUTTO il file in una stringa sola (Giorno 02 §5.1).
print(type(testo))

# 2. .replace() rende visibile l'invisibile: ogni \n diventa un'etichetta.
print(testo.replace("\n", "[a capo]"))

# 3. len() conta anche gli a capo: 3 codici da 8 caratteri + 3 a capo = 27.
print("Caratteri:", len(testo))

# 4. Le "righe" sono il numero di a capo, e .count() li conta (Giorno 02).
print("Righe:", testo.count("\n"))

# TRABOCCHETTO: print(testo) da solo non vi mostra nessun \n: li ESEGUE,
# andando a capo. Per questo sembra che non ci siano, e per questo al
# capitolo 5 un confronto fallirà per un carattere che non vedete.

# <class 'str'>
# LS123455[a capo]LS400004[a capo]LS314154[a capo]
# Caratteri: 27
# Righe: 3
```

Un file di testo non contiene righe: contiene caratteri, e fra questi il \n che dice "qui si va a capo". Ogni riga letta da un file si porta dietro il suo \n. Su Windows l'a capo sul disco è di due caratteri, ma Python li traduce da solo in un \n singolo.

2.3 La codifica: encoding="utf-8" sempre

Se scrivete...	Che cosa succede
<code>open(nome, "r")</code> senza encoding	funziona sul vostro computer, forse non su quello del collega
<code>open(nome, "r", encoding="utf-8")</code>	funziona su tutti e due
il file arriva da un vecchio gestionale	può non essere UTF-8: si chiede a chi lo manda

Il disco conserva numeri, non lettere: la CODIFICA è la tabella che dice quale numero è quale carattere. Senza encoding=... Python usa quella del sistema operativo: su Mac UTF-8, su un Windows italiano di solito no. Regola del corso: encoding="utf-8" in ogni open, senza eccezioni.

2.3 Andata e ritorno con la stessa codifica

```
# Scrittura e lettura con la STESSA codifica dichiarata: le accentate
# fanno il viaggio andata e ritorno intatte.

with open("accenti.txt", "w", encoding="utf-8") as f:

    f.write("Città di Villanova\n")

    f.write("perché\n")


with open("accenti.txt", "r", encoding="utf-8") as f:

    testo = f.read()


print(testo, end="")

# len() conta i CARATTERI: 18 + 1 a capo, 6 + 1 a capo.
print("Caratteri letti:", len(testo))


# TRABOCCHETTO: lo stesso programma senza encoding=... gira identico su un
# Mac e potrebbe darvi lettere storpiate su Windows. Il difetto non si
# vede sulla macchina di chi scrive il codice: si vede su quella di chi
# lo usa.


# Città di Villanova
# perché
# Caratteri letti: 26
```

2.3 CHICCA: "perché" è lunga 6 caratteri e 7 byte

```
parola = "perché"

# 1. len() su una stringa: CARATTERI, come nel Giorno 02.
print("Caratteri:", len(parola))

# 2. .encode() trasforma il testo nei BYTE che vanno sul disco. Il risultato
# non e' una stringa: e' un tipo nuovo, bytes, con la b davanti.
byte = parola.encode("utf-8")
print("Byte in UTF-8:", len(byte))
print(byte)

# TRABOCCHETTO: senza accento i due conti coincidono, e il problema sparisce.
# E' per questo che un programma collaudato solo su parole inglesi sembra
# perfetto fino al primo cognome come Nicolò.
print("Byte di 'perche':", len("perche".encode("utf-8")))

# Caratteri: 6
# Byte in UTF-8: 7
# b'perch\xc3\xa9'
# Byte di 'perche': 6
```

2.4 La codifica sbagliata, primo esito: lettere storpiate

```
# 1. Scrittura corretta, in UTF-8.

with open("rubrica.txt", "w", encoding="utf-8") as f:

    f.write("perché\n")

    f.write("Nicolò\n")


# 2. Lettura con la tabella SBAGLIATA: latin-1, quella dei vecchi sistemi.

with open("rubrica.txt", "r", encoding="latin-1") as f:

    testo = f.read()


print(testo, end="")

print("Caratteri letti:", len(testo))


# TRABOCCHETTO: nessun errore, nessun avviso. latin-1 accetta QUALSIASI
# byte, quindi legge i due byte della é come due lettere diverse. Il
# programma prosegue felice con dati corrotti: 16 caratteri invece di 14,
# e un cognome che non combacera' piu' con nessuna ricerca.


# perchÃ©
# NicolÃ²

# Caratteri letti: 16
```

2.4 Secondo esito: UnicodeDecodeError

```
with open("vecchio.txt", "w", encoding="cp1252") as f:  
    f.write("perché\n")
```

```
with open("vecchio.txt", "r", encoding="utf-8") as f:  
    testo = f.read()  
print(testo)
```

Traceback (most recent call last):

File "prova.py", line 5, in <module>

testo = f.read()

File "<frozen codecs>", line 325, in decode

UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe9 in position 5: invalid continuation byte

Dal basso: UTF-8 non decifra il byte 0xe9 alla posizione 5 (p, e, r, c, h, e poi lui). <frozen codecs> è codice interno di Python. TRABOCCHETTO: la riga indicata è il read(), ma il difetto sta nel file, cioè nei dati: il codice è giusto.

2.4 I due esiti a confronto: storpiato o fermo

Esito	Che cosa vedete	Quanto è pericoloso
lettere storpiate (perchÃ©)	niente errore, dati corrotti	molto: ve ne accorgete tardi
UnicodeDecodeError	il programma si ferma subito	poco: sapete dove guardare

In cp1252 la é è UN byte solo, 0xe9: per UTF-8 quel byte annuncia una lettera lunga, e il byte dopo non è quello atteso. La difesa di oggi è una: encoding="utf-8" sempre, e per un file che arriva da fuori si chiede a chi lo manda in che codifica è scritto.

3. APRIRE E CHIUDERE UN FILE:

with open()

Cinque pezzi, quattro modalità, e il traceback del file che non c'è

3.1 Anatomia di with open(...) as f: i cinque pezzi

```
# I cinque pezzi, da sinistra a destra:

# with          apri, e CHIUDI da solo quando il blocco rientrato finisce

# open(...)     la funzione che apre: il primo argomento e' il percorso

# "w"          la MODALITA': che cosa volete farci (paragrafo 3.3)

# encoding=...  la tabella caratteri-byte del capitolo 2, sempre utf-8

# as f          il nome con cui chiamate il file aperto dentro il blocco

with open("sportello.txt", "w", encoding="utf-8") as f:

    # Rientro di quattro spazi: questa e' la zona in cui il file e' aperto.

    f.write("Serra;anagrafe\n")

    f.write("Rinaldi;tributi\n")

# Qui il file e' gia' chiuso, e il suo contenuto e' sul disco. Riaprirlo in

# lettura con un nome diverso da f funziona uguale: il nome e' una scelta.

with open("sportello.txt", "r", encoding="utf-8") as registro:

    print(registro.read(), end="")

# TRABOCCHETTO: i due punti in fondo alla riga del with sono obbligatori,

# come in un if o in un def. Senza, SyntaxError prima ancora di partire.

# Serra;anagrafe

# Rinaldi;tributi
```

Intestazione con i due punti, corpo rientrato: come un if e un def. Nel with il corpo è LA PARTE IN CUI IL FILE È APERTO. La modalità si può omettere (il predefinito è "r"), ma nel corso si scrive: dichiara l'intenzione a chi legge.

3.2 Perché with: close() a mano e l'attributo .closed

```
# 1. Il modo "a mano": open() consegna il file aperto, close() lo chiude.

# L'attributo .closed risponde True o False: il file e' chiuso?

f = open("manuale.txt", "w", encoding="utf-8")

f.write("riga scritta a mano\n")

print("Prima di close():", f.closed)

f.close()

print("Dopo close():", f.closed)


# 2. Con with la chiusura e' automatica all'uscita dal blocco rientrato.

with open("automatico.txt", "w", encoding="utf-8") as g:

    g.write("riga scritta con with\n")

    print("Dentro il with:", g.closed)

print("Fuori dal with:", g.closed)


# TRABOCCHETTO: dimenticare f.close() non da' nessun errore. Il programma
# finisce, e quasi sempre Python chiude il file uscendo. Quasi sempre:
# e' esattamente il tipo di difetto che si presenta una volta al mese.


# Prima di close(): False

# Dopo close(): True

# Dentro il with: False

# Fuori dal with: True
```

Un file aperto è una risorsa prestata dal sistema operativo: finché non lo chiudete, quello che avete scritto può restare in sospeso (6.6), e su Windows il file resta bloccato. Chiudere a mano si può: il problema è ricordarsene SEMPRE.

3.2 Il return a metà del with: il file si chiude lo stesso

```
with open("sportello.txt", "w", encoding="utf-8") as f:

    f.write("Serra;anagrafe\n")

    f.write("Rinaldi;tributi\n")


def leggi_tutto(percorso):

    # Restituisce il testo del file e l'oggetto file, per ispezionarlo.

    with open(percorso, "r", encoding="utf-8") as f:

        testo = f.read()

        # return a META' del with: la funzione esce subito, e il with chiude

        # il file uscendo, prima ancora di consegnare i due valori.

        return testo, f


# Due valori restituiti e spaccettati, come nel Giorno 06 $10.1.
testo, file_usato = leggi_tutto("sportello.txt")

print("Caratteri letti:", len(testo))

print("Il file è chiuso dopo il return?", file_usato.closed)


# TRABOCCHETTO: con open() e close() a mano, il return prima del close()

# lascerebbe il file aperto: la riga del close() non verrebbe mai

# raggiunta. Nessun errore, e un file che resta bloccato.


# Caratteri letti: 31

# Il file è chiuso dopo il return? True
```

3.3 Le modalità r, w, a, x: il file c'è o non c'è?

Modalità	Nome	Se il file NON esiste	Se il file ESISTE
"r"	lettura	si ferma: FileNotFoundError	lo legge, non lo tocca
"w"	scrittura	lo crea vuoto	LO SVUOTA e riparte da zero
"a"	aggiunta	lo crea vuoto	scrive IN CODA, il resto resta
"x"	creazione esclusiva	lo crea vuoto	si ferma: FileExistsError

La modalità è una lettera, e la domanda che decide è una sola: il file esiste già? Nella colonna di destra si perdono i dati: "w" distrugge senza chiedere.

3.3 w crea o svuota, a accoda: la prova

```
NOME = "coda.txt"

# 1. "w": il file nasce (o viene SVUOTATO, se c'era) con due righe.

with open(NOME, "w", encoding="utf-8") as f:

    f.write("Serra\n")

    f.write("Rinaldi\n")

# 2. "a": aggiunge IN CODA, senza toccare quello che c'e'.

with open(NOME, "a", encoding="utf-8") as f:

    f.write("Motta\n")

# .split() senza argomenti (Giorno 03 $7.6): le righe una accanto all'altra.

with open(NOME, "r", encoding="utf-8") as f:

    print("Dopo w + a:", f.read().split())


# 3. "w" una seconda volta: il contenuto di prima sparisce, SENZA avviso.

with open(NOME, "w", encoding="utf-8") as f:

    f.write("Barbato\n")

with open(NOME, "r", encoding="utf-8") as f:

    print("Dopo un altro w:", f.read().split())


# TRABOCCHETTO: il file si svuota nell'istante in cui open(..., "w") viene
# eseguito, non alla prima write(). Aprire in "w" e non scrivere niente
# lascia un file vuoto.
```

Serra, Rinaldi e Motta c'erano, e il secondo "w" li ha cancellati senza un avviso.

```
# Dopo w + a: ['Serra', 'Rinaldi', 'Motta']

# Dopo un altro w: ['Barbato']
```

```
NOME = "report_consegnato.txt"

with open(NOME, "x", encoding="utf-8") as f:
    f.write("Report di chiusura\n")
print("Primo tentativo: file creato")

with open(NOME, "x", encoding="utf-8") as f:
    f.write("Report rifatto\n")
print("Secondo tentativo: file creato")

Primo tentativo: file creato
Traceback (most recent call last):
  File "prova.py", line 7, in <module>
    with open(NOME, "x", encoding="utf-8") as f:
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
FileExistsError: [Errno 17] File exists: 'report_consegnato.txt'
```

Dal basso: che cosa (FileExistsError), quale file, dove (riga 7). [Errno 17] è il codice del sistema operativo: non vi serve. TRABOCCHETTO: con "w" al posto del secondo "x" nessun errore, e il report consegnato sostituito da quello rifatto. Al paragrafo 8.4 lo fa un if.

3.4 FileNotFoundError letto dal basso: il percorso cercato

```
from pathlib import Path
```

```
percorso = Path("dati") / "ordini_ieri.txt"
with open(percorso, "r", encoding="utf-8") as f:
    print(f.read())
```

```
Traceback (most recent call last):
```

```
File "prova.py", line 4, in <module>
```

```
    with open(percorso, "r", encoding="utf-8") as f:
```

```
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
FileNotFoundError: [Errno 2] No such file or directory: 'dati/ordini_ieri.txt'
```

Il tipo, la spiegazione e soprattutto IL PERCORSO ESATTO CHE PYTHON HA CERCATO: leggetelo ad alta voce, risolve il novanta per cento dei casi. TRABOCCHETTO: il messaggio non dice "il file non esiste", dice "non c'è DOVE l'ho cercato". Su Windows esce con le barre rovesciate.

3.4 Il traceback a due livelli: dove scoppia, chi ha chiamato

```
4 def conta_righe(percorso):
5     with open(percorso, "r", encoding="utf-8") as f:
6         return f.read().count("\n")
9 print("ticket.txt:", conta_righe(Path("dati") / "ticket.txt"))
10 vecchi = Path("dati") / "ticket_vecchi.txt"
11 print("ticket_vecchi.txt:", conta_righe(vecchi))
```

ticket.txt: 30

Traceback (most recent call last):

```
File "prova.py", line 11, in <module>
    print("ticket_vecchi.txt:", conta_righe(vecchi))
                                ^^^^^^^^^^^^^^^^^^^
```

```
File "prova.py", line 5, in conta_righe
    with open(percorso, "r", encoding="utf-8") as f:
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

FileNotFoundError: [Errno 2] No such file or directory: 'dati/ticket_vecchi.txt'

Il blocco File più in basso è dove è scoppiato, il più in alto è chi l'ha chiamato. TRABOCCHETTO: l'errore scoppia alla riga 5, dentro la funzione, ma la causa è alla riga 10, dove si sceglie il nome. La funzione è giusta: ha ricevuto un percorso sbagliato.

3.5 rb e wb: i file che non sono testo, in un paragrafo

```
# "wb": scrittura binaria, senza encoding, perche' i byte vanno sul disco

# cosi' come sono. La b davanti alle virgolette dice "questi sono byte, non testo".

with open("etichetta.bin", "wb") as f:

    f.write(b"LS\x00\x01")


# "rb": lettura binaria. read() consegna un oggetto bytes, non una str.

with open("etichetta.bin", "rb") as f:

    contenuto = f.read()


print(type(contenuto))

print(contenuto)

print("Byte:", len(contenuto))


# TRABOCCHETTO: l'indice su un bytes non da' un carattere, da' un NUMERO.

#   contenuto[0] e' 76, il codice della L. Su una stringa sarebbe "L".

print("Primo byte:", contenuto[0])


# <class 'bytes'>

# b'LS\x00\x01'

# Byte: 4

# Primo byte: 76
```

Un'immagine, un PDF, un file Excel non sono testo: aperti con encoding="utf-8" danno l'UnicodeDecodeError del 2.4. È un cenno: nessun esercizio di oggi apre un file binario. Un .xlsx NON è testo, e oggi la strada è chiederne l'esportazione in testo.

4. LEGGERE UN FILE

Tutto insieme, a righe, una riga alla volta: quattro modi e un segnalibro

4.1 read(): tutto il file in una stringa sola

```
from pathlib import Path

PERCORSO = Path("dati") / "lettera_candidatura.txt"

# read() consegna TUTTO il file in una stringa sola: comodo quando il file e'
# piccolo e vi serve per intero, come una lettera.

with open(PERCORSO, "r", encoding="utf-8") as f:

    testo = f.read()

# 1. Caratteri: len() sulla stringa intera, a capo compresi (capitolo 2.2).
print("Caratteri:", len(testo))

# 2. Righe: gli a capo, contati con .count().
print("Righe:", testo.count("\n"))

# 3. Parole: .split() senza argomenti taglia su spazi E a capo, e non crea
# pezzi vuoti per gli spazi doppi (Giorno 03 $7.6).

parole = testo.split()

print("Parole:", len(parole))

print("Prima e ultima:", parole[0], "/", parole[-1])

# TRABOCCHETTO: testo.split(" ") con lo spazio esplicito sembra uguale e non
# lo e': non taglia sugli a capo, quindi "TalentHub,\nvi" resta una parola
# sola, e ogni spazio in coda a una riga crea una "parola" vuota.

# Caratteri: 560
# Righe: 12
# Parole: 85
# Prima e ultima: Gentile / Ferri
```

4.2 readlines(): una lista di righe, ognuna con il suo \n

```
from pathlib import Path

with open(Path("dati") / "lettera_candidatura.txt", "r", encoding="utf-8") as f:
    righe = f.readlines()

# 1. Il tipo e' list, e len() conta le RIGHE (Giorno 07 $9.1).
print(type(righe))

print("Elementi:", len(righe))

# 2. Indice 0 e indice -1, come nel Giorno 07 $3.1. Il .replace() mostra il
#     \n che ogni elemento si porta dietro.
print("Prima:", righe[0].replace("\n", "[a capo]"))
print("Ultima:", righe[-1].replace("\n", "[a capo]"))

# 3. La quinta riga del file e' la riga vuota fra i due paragrafi.
print("Quinta:", righe[4].replace("\n", "[a capo]"))

# TRABOCCHETTO: la riga vuota NON e' la stringa vuota "". E' "\n", lunga
#     uno. Un controllo scritto riga == "" non la riconoscerà mai: vedi 5.3.

# <class 'list'>
# Elementi: 12
# Prima: Gentile ufficio selezione di TalentHub,[a capo]
# Ultima: Giulia Ferri[a capo]
# Quinta: [a capo]
```

4.3 for riga in f: una riga alla volta, il modo che regge

```
from pathlib import Path

# Contatori azzerati FUORI dal ciclo: il Giorno 05 §6.4.

totali = 0

urgenti = 0

anche_credenziali = 0

with open(Path("dati") / "ticket.txt", "r", encoding="utf-8") as f:

    # Il for su un file aperto consegna UNA RIGA a ogni giro, già con il suo

    # \n. E' l'iterazione diretta del Giorno 07 §6.1: niente indici.

    for riga in f:

        totali += 1

        testo = riga.lower()

        if "urgente" in testo:

            urgenti += 1

            if "credenziali" in testo:

                anche_credenziali += 1

print("Ticket letti:", totali)

print("Con 'urgente':", urgenti, "- di cui sulle credenziali:", anche_credenziali)

if totali > 0:

    print(f"Quota urgenti: {urgenti / totali:.1%}")

# TRABOCCHETTO: senza .lower(), "urgente" in riga distingue le maiuscole, e

# un ticket scritto "URGENTE, rispondete" sfuggirebbe al conteggio senza

# errori. E riga.lower() da solo, senza assegnarlo, non cambia niente: le

# stringhe non si modificano (Giorno 02 §9.4).
```

4.4 readline() e la stringa vuota di fine file: la sentinella

```
from pathlib import Path

lette = 0
vuote = 0

with open(Path("dati") / "codici_colli.txt", "r", encoding="utf-8") as f:

    # 1. Lettura PRIMA del ciclo: il while ha bisogno di un valore da
    #    controllare gia' al primo giro (i tre obblighi, Giornata 05 $7.2).
    riga = f.readline()

    while riga != "":

        lette += 1

        # Una riga vuota DENTRO il file vale "\n", non "": non ferma il ciclo.

        if riga == "\n":

            vuote += 1

        # 2. Lettura IN CODA al corpo: senza, il while gira per sempre.
        riga = f.readline()

print("Righe lette:", lette)

print("Di cui vuote:", vuote)

print("Lunghezza dell'ultima lettura:", len(riga))

# TRABOCCHETTO: se la condizione fosse riga != "\n", il ciclo si
#    fermerebbe alla prima riga vuota in mezzo al file (la settimana), e le
#    ultime sette righe non verrebbero mai lette. Nessun errore.
```

4.4 readline() per l'intestazione, poi il for sul resto

```
from pathlib import Path

with open(Path("dati") / "spedizioni.txt", "r", encoding="utf-8") as f:
    # La prima riga e' un'intestazione, non un dato: si legge a parte.
    intestazione = f.readline()

    # Il for riparte dalla SECONDA riga: il file ricorda dove siete arrivati.
    resto = 0

    for riga in f:
        resto += 1

# end="" perche' l'intestazione ha gia' il suo \n (Giorno 01 $10.4).
print("Intestazione:", intestazione, end="")
print("Righe dopo l'intestazione:", resto)

# TRABOCCHETTO: senza end="" l'output avrebbe una riga vuota in mezzo: il
# \n dell'intestazione piu' quello della print. Un a capo di troppo e' il
# primo indizio che una riga si porta dietro il suo \n.

# Intestazione: # export magazzino LogiSud - codice;zona;peso_kg
# Righe dopo l'intestazione: 26
```

4.5 Il file si legge una volta sola: il secondo for a zero giri

```
from pathlib import Path

PERCORSO = Path("dati") / "codici_colli.txt"

with open(PERCORSO, "r", encoding="utf-8") as f:

    primo_giro = 0

    for riga in f:

        primo_giro += 1

    # 1. Secondo for sullo STESSO file aperto: il segno e' gia' in fondo.

    secondo_giro = 0

    for riga in f:

        secondo_giro += 1

print("Sul file - primo giro:", primo_giro, "- secondo giro:", secondo_giro)

# TRABOCCHETTO: il secondo giro sul file fa ZERO giri senza errori, come il

# ciclo a zero giri del Giorno 05 $7.3.
```

Un file aperto ha un SEGNO DI LETTURA, come il segnalibro in un libro: ogni lettura lo sposta avanti, e arrivato in fondo resta in fondo. Il secondo for non trova niente da leggere, e non protesta.

```
# Sul file - primo giro: 14 - secondo giro: 0
```

4.5 Il rimedio: caricare una volta in una lista

```
# 2. Il rimedio: caricare UNA volta in una lista, che non ha segnalibro e si
#     scorre quante volte serve (Giorno 07 §6.1).
with open(PERCORSO, "r", encoding="utf-8") as f:
    righe = f.readlines()

con_prefisso = 0
for riga in righe:
    if riga.startswith("LS"):
        con_prefisso += 1

non_vuote = 0
for riga in righe:
    if riga != "\n":
        non_vuote += 1

print("Sulla lista - iniziano con LS:", con_prefisso, "- non vuote:", non_vuote)

# TRABOCCHETTO: sulla lista, 10 e non 12: "ls900016" e' minuscolo e
#     " LS000000 " comincia con due spazi. Tutti e due sono problemi del
#     capitolo 5.

# Sulla lista - iniziano con LS: 10 - non vuote: 13
```

4.5 CHICCA: f.seek(0) riavvolge, f.tell() dice dove siete

```
with open("tre_codici.txt", "w", encoding="utf-8") as f:
    f.write("LS123455\nLS400004\nLS314154\n")

with open("tre_codici.txt", "r", encoding="utf-8") as f:
    # 1. Appena aperto, il segno e' all'inizio: posizione zero.
    print("All'apertura:", f.tell())

    # 2. Una riga sono 8 caratteri piu' l'a capo: il segno avanza di 9.
    f.readline()

    print("Dopo una riga:", f.tell())

    # 3. read() porta il segno in fondo: 27, la lunghezza del capitolo 2.2.
    f.read()

    print("A fine file:", f.tell())

    # 4. seek(0) lo riporta all'inizio, e si rilegge senza riaprire.
    f.seek(0)

    print("Dopo seek(0):", f.tell())

    print("Di nuovo la prima:", f.readline(), end="")

# TRABOCCHETTO: il numero di tell() conta i BYTE, non i caratteri. Su un
# file con le accentate i due conti divergono (la CHICCA del 2.3), e
# seek() va usato solo con 0 o con un valore restituito da tell().

# All'apertura: 0
# Dopo una riga: 9
# A fine file: 27
# Dopo seek(0): 0
# Di nuovo la prima: LS123455
```

Il segnalibro esiste davvero, ed è un numero: tell() lo legge, seek(0) lo riporta all'inizio. È il secondo for a zero giri, visto dall'interno.

4.6 Quale dei quattro scegliere: tabella di decisione

Che cosa vi serve	Metodo	Perché
il testo intero, per contare o cercare	<code>f.read()</code>	una stringa sola, file piccoli
le righe con un indice (la terza, l'ultima)	<code>f.readlines()</code>	una lista, si scorre più volte
la sola intestazione, poi il resto	<code>f.readline() + for</code>	legge una riga e si ferma
elaborare ogni riga, anche su file enormi	<code>for riga in f</code>	una riga in memoria per volta

La scelta segue la domanda, come nel Giorno 05 §11.1 fra `for` e `while`. In caso di dubbio: `for riga in f`.

4.6 Lo stesso file letto nei quattro modi

```
NOME = "zone.txt"

with open(NOME, "w", encoding="utf-8") as f:

    f.write("NORD\nCENTRO\nSUD\n")

# Lo stesso file letto nei quattro modi. Ogni with riparte dall'inizio,
# perche' ogni open() crea un segno di lettura nuovo.

with open(NOME, "r", encoding="utf-8") as f:

    print("read()      -> una stringa di", len(f.read()), "caratteri")

with open(NOME, "r", encoding="utf-8") as f:

    print("readlines() -> una lista di", len(f.readlines()), "righe")

with open(NOME, "r", encoding="utf-8") as f:

    print("readline()   -> la sola prima riga:", f.readline(), end="")

with open(NOME, "r", encoding="utf-8") as f:

    giri = 0

    for riga in f:

        giri += 1

    print("for riga in f ->", giri, "giri, una riga alla volta")


# TRABOCCHETTO: readline() e readlines() differiscono per una lettera e
# restituiscono due tipi diversi: una str e una list. Confonderli da'
# len() sbagliati senza errori: 5 (i caratteri di "NORD\n") contro 3.

# read()      -> una stringa di 16 caratteri
# readlines() -> una lista di 3 righe
# readline()  -> la sola prima riga: NORD
# for riga in f -> 3 giri, una riga alla volta
```

5. IL FINE RIGA E LA PULIZIA DELLE RIGHE

Il carattere che non vedete, e i confronti che non trovano niente

5.1 Il \n che non si vede e fa fallire il confronto

```
with open("presenze.txt", "w", encoding="utf-8") as f:
    f.write("Rinaldi\nSerra\nMotta\n")

# 1. La ricerca "ovvia": una riga uguale al cognome cercato (Giorno 04).
trovati = 0

with open("presenze.txt", "r", encoding="utf-8") as f:
    for riga in f:
        if riga == "Serra":
            trovati += 1

print("Serra trovata:", trovati, "volte")

# 2. Il perche', misurato: la riga letta e' "Serra\n", sei caratteri.
with open("presenze.txt", "r", encoding="utf-8") as f:
    righe = f.readlines()

print("Lunghezza:", len(righe[1]), "contro", len("Serra"))
print("Con l'a capo visibile:", righe[1].replace("\n", "[a capo]"))

# TRABOCCHETTO: e' il sintomo del Giorno 05 $14.5, il contatore che resta a
# zero senza errori. La' la causa era l'azzeramento dentro il ciclo; qui
# e' un carattere che print() non vi mostra.

# Serra trovata: 0 volte
# Lunghezza: 6 contro 5
# Con l'a capo visibile: Serra[a capo]
```

5.2 .rstrip("\n") contro .strip(): il minimo indispensabile

```
# Una riga vera di spedizioni.txt: spazi ovunque e zona in minuscolo.
riga = " LS-2026-MI-0999 ; nord ; 5.0 \n"

# 1. .rstrip("\n") toglie SOLO l'a capo in coda. Le quadre mostrano i bordi.
print "[" + riga.rstrip("\n") + "]"

# 2. .strip() sulla riga intera toglie gli spazi ai DUE estremi...
pulita = riga.strip()
print "[" + pulita + "]"

# 3. ...ma quelli attorno ai ; stanno DENTRO la riga, e finiscono nei campi.
codice, zona, peso = pulita.split(";")
print "[" + zona + "]", zona == "NORD"

# 4. La pulizia vera: campo per campo, dopo lo split.
zona = zona.strip().upper()
print "[" + zona + "]", zona == "NORD"

# TRABOCCHETTO: " nord " non e' una zona del listino. Senza il punto 4,
# la ricerca nel dizionario del Giorno 07 darebbe KeyError, oppure,
# con .get(), il ripiego: una spedizione "gratis" senza nessun errore.

# [ LS-2026-MI-0999 ; nord ; 5.0 ]
# [LS-2026-MI-0999 ; nord ; 5.0]
# [ nord ] False
# [NORD] True
```

5.2 Perché non .strip() sulla riga: la tabulazione

```
# Terzo campo VUOTO: la riga finisce con una tabulazione e poi l'a capo.
riga = "LS-2026-MI-0042\tNORD\t\n"

# Togliendo solo l'a capo, il \t finale resta e il campo vuoto c'e'.
campi = riga.rstrip("\n").split("\t")
print("Togliendo solo l'a capo:", len(campi), "campi")

# .strip() considera il \t uno spazio da togliere: il terzo campo sparisce.
campi = riga.strip().split("\t")
print("Con .strip() sulla riga:", len(campi), "campi")

# TRABOCCHETTO: il campo sparito non da' errore qui. Lo da' tre righe dopo,
#   quando l'unpacking a tre nomi trova due pezzi: il sintomo lontano dalla
#   causa del Giorno 06 §6.2.

# Togliendo solo l'a capo: 3 campi
# Con .strip() sulla riga: 2 campi
```

5.3 Righe vuote e righe di commento: saltarle con continue

```
from pathlib import Path

utili = 0

saltate = 0

with open(Path("dati") / "spedizioni.txt", "r", encoding="utf-8") as f:

    for riga in f:

        # 1. Prima istruzione: via l'a capo (5.2).

        riga = riga.rstrip("\n")

        # 2. Riga vuota (o di soli spazi) OPPURE commento: si salta. .strip()

        #     qui serve solo al controllo, la riga resta com'era.

        if riga.strip() == "" or riga.startswith("#"):

            saltate += 1

            continue

        # 3. Da qui in giu' arrivano solo righe con dei dati.

        utili += 1

print("Righe utili:", utili)

print("Righe saltate:", saltate)

# TRABOCCHETTO: con break al posto di continue le righe utili sarebbero 0.

# La PRIMA riga del file e' un commento, e break chiude il ciclo al primo

# giro. Nessun errore: un file di 27 righe che sembra vuoto.

# Righe utili: 25

# Righe saltate: 2
```

5.4 Normalizzare prima di confrontare: tutti e due i lati

```
from pathlib import Path

CERCATO = "rinaldi"

trovati_grezzo = 0

trovati_pulito = 0

with open(Path("dati") / "anagrafiche.txt", "r", encoding="utf-8") as f:

    for riga in f:

        riga = riga.rstrip("\n")

        if riga.strip() == "" or riga.startswith("#"):

            continue

        # Lo split con unpacking del Giorno 03: nome;cognome;email.

        nome, cognome, email = riga.split(";")

        # 1. Confronto grezzo: " Rinaldi " contro "rinaldi". Mai uguali.

        if cognome == CERCATO:

            trovati_grezzo += 1

        # 2. Confronto normalizzato: .strip().lower() su ENTRAMBI i lati.

        if cognome.strip().lower() == CERCATO.strip().lower():

            trovati_pulito += 1

            print("Trovato:", nome.strip().title(), cognome.strip().title(),

                  email.strip().lower())

print("Confronto grezzo:", trovati_grezzo)

print("Confronto normalizzato:", trovati_pulito)

# TRABOCCHETTO: normalizzare solo il dato letto non basta. Se chi cerca

#  digita "RINALDI", il lato destro resta maiuscolo e il confronto torna a

#  zero: per questo anche CERCATO passa da .strip().lower().
```

5.4 La normalizzazione scopre i doppioni

```
ELENCO = ["MARTA.SERRA@example.it", "marta.serra@example.it ",
          "Paolo.Ferraro@example.it", "paolo.ferraro@example.it"]

visti_grezzi = []
visti_puliti = []

for email in ELENCO:

    # 1. Senza pulizia: quattro stringhe diverse, quattro "contatti".

    if email not in visti_grezzi:

        visti_grezzi.append(email)

    # 2. Con la pulizia: la chiave di confronto e' la forma normalizzata.

    chiave = email.strip().lower()

    if chiave not in visti_puliti:

        visti_puliti.append(chiave)

print("Diversi per Python:", len(visti_grezzi))

print("Diversi dopo la pulizia:", len(visti_puliti))

# TRABOCCHETTO: il doppione nascosto non da' mai errore. Da' una rubrica con
# la stessa persona due volte, e un invio di posta che arriva doppio.

# Diversi per Python: 4
# Diversi dopo la pulizia: 2
```

5.5 L'ultima riga senza \n e la riga vuota in fondo

```
with open("senza_finale.txt", "w", encoding="utf-8") as f:
    f.write("Serra\nMotta")          # l'ultima riga NON ha l'a capo

with open("vuota_in_coda.txt", "w", encoding="utf-8") as f:
    f.write("Serra\nMotta\n\n")      # una riga vuota in fondo

for nome in ["senza_finale.txt", "vuota_in_coda.txt"]:
    with open(nome, "r", encoding="utf-8") as f:
        righe = f.readlines()

        # 1. La pulizia standard di 5.2 e 5.3, su tutte e due le liste.
        utili = 0

        for riga in righe:
            riga = riga.rstrip("\n")

            if riga.strip() == "" or riga.startswith("#"):
                continue

            utili += 1

        # 2. readlines() vede due file diversi, la pulizia li rende uguali.
        print(f"{nome:<18}readlines(): {len(righe)}    utili: {utili}")

# TRABOCCHETTO: senza .rstrip("\n"), nel primo file "Motta" == "Motta"
# sarebbe vero e "Serra" == "Serra" falso. Un confronto che funziona solo
# sull'ultima riga e' il sintomo piu' difficile da capire della giornata.

# senza_finale.txt readlines(): 2    utili: 2
# vuota_in_coda.txt readlines(): 3    utili: 2
```

5.5 CHICCA: split("\n") inventa una riga, splitlines() no

```
testo = "Serra\nMotta\n"
```

```
# split("\n") taglia A OGNI \n: dopo l'ultimo resta una stringa vuota.
```

```
print(testo.split("\n"))
```

```
# splitlines() tratta l'ultimo \n come fine riga, non come riga nuova.
```

```
print(testo.splitlines())
```

```
# TRABOCCHETTO: una riga vuota IN MEZZO resta anche con splitlines(), perche'
```

```
# e' una riga vera del file. Il continue del 5.3 serve lo stesso.
```

```
print("Serra\n\nMotta\n".splitlines())
```

```
# ['Serra', 'Motta', '']
```

```
# ['Serra', 'Motta']
```

```
# ['Serra', '', 'Motta']
```

6. SCRIVERE UN FILE

write, print con file=, writelines, e la lettera che cancella

6.1 write() non va a capo e restituisce i caratteri scritti

```
with open("uscita.txt", "w", encoding="utf-8") as f:

    # 1. Tre write() senza \n: finiscono tutte sulla stessa riga.

    f.write("Serra")

    f.write("Rinaldi")

    # 2. write() RESTITUISCE quanti caratteri ha scritto. Finora il valore

    #   si ignorava; si puo' anche catturare, come nel Giorno 06 §5.1.

    scritti = f.write("Motta\n")

    accentata = f.write("perché\n")
```

```
with open("uscita.txt", "r", encoding="utf-8") as f:

    print(f.read(), end="")

print("Restituito da write('Motta\n'):", scritti)

print("Restituito da write('perché\n'):", accentata)

# TRABOCCHETTO: il numero conta i CARATTERI, non i byte. "perché\n" vale 7,

#   anche se sul disco occupa 8 byte (la CHICCA del 2.3).

# SerraRinaldiMotta

# perché

# Restituito da write('Motta\n'): 6

# Restituito da write('perché\n'): 7
```

6.1 write() accetta solo stringhe: TypeError

```
colli = 42
with open("conteggio.txt", "w", encoding="utf-8") as f:
    f.write(colli)
```

```
Traceback (most recent call last):
  File "prova.py", line 3, in <module>
    f.write(colli)
TypeError: write() argument must be str, not int
```

print(colli) avrebbe funzionato: print converte da sola, write no. È un errore di chi scrive il codice, non dei dati: si corregge convertendo prima di scrivere.
TRABOCCHETTO: conteggio.txt esiste comunque, VUOTO. L'open() in "w" lo ha già creato.

6.2 Scrivere numeri: prima la f-string, poi write()

```
SPEDIZIONI = [("LS-2026-MI-0042", "NORD", 12.5),
               ("LS-2026-NA-0913", "SUD", 6.7),
               ("LS-2026-CA-1806", "ISOLE", 36.4)]

with open("report_colli.txt", "w", encoding="utf-8") as f:

    # 1. L'intestazione con le STESSE larghezze delle righe (Giorno 07 $14.1).
    f.write(f'{ "CODICE":<18}{ "ZONA":<8}{ "KG":>6}\n')

    # 2. Unpacking della tupla nel for, e ogni numero passa dalla f-string:
    #      :>6.1f = a destra, sei colonne, un decimale. Il \n lo mettete voi.
    for codice, zona, peso in SPEDIZIONI:
        f.write(f'{codice:<18}{zona:<8}{peso:>6.1f}\n')

with open("report_colli.txt", "r", encoding="utf-8") as f:
    print(f.read(), end="")

# TRABOCCHETTO: f.write(str(peso) + "\n") funziona, ma scrive il float così'
#   com'e': 12.5 e 6.7 vanno bene, un 0.1 + 0.2 finirebbe nel file come
#   0.300000000000000004 (Giorno 02 $7.2). La f-string decide per voi.

# CODICE      ZONA      KG
# LS-2026-MI-0042  NORD      12.5
# LS-2026-NA-0913  SUD        6.7
# LS-2026-CA-1806  ISOLE     36.4
```

6.3 print(..., file=f): la print che conoscete, verso il disco

```
with open("scontrino.txt", "w", encoding="utf-8") as f:

    # 1. file=f e' un argomento nominato (Giorno 06 §9.1): va dopo gli altri.

    print("NovaStore - scontrino", file=f)

    # 2. Il numero 3 NON va convertito: print lo fa da sola, write no.

    print("Articoli:", 3, file=f)

    # 3. sep ed end funzionano identici anche verso il file.

    print("Totale", 126.8, "euro", sep=" | ", file=f)

    print("Grazie", end="!\n", file=f)
```

```
with open("scontrino.txt", "r", encoding="utf-8") as f:

    print(f.read(), end="")
```

```
# TRABOCCHETTO: dimenticare file=f non da' errore. La riga finisce sullo
# schermo invece che nel file, e il file resta con una riga in meno.
```

```
# NovaStore - scontrino
# Articoli: 3
# Totale | 126.8 | euro
# Grazie!
```

6.3 write() o print(file=f): quale usare

Vi serve...	Usate	Perché
una riga di testo con numeri e spazi, come a video	<code>print(..., file=f)</code>	va a capo e converte da sola
scrivere esattamente una stringa, senza aggiunte	<code>f.write(s)</code>	nessuna sorpresa, controllo totale
una colonna allineata	tutte e due, con la f-string	è la f-string che allinea

6.4 writelines() non aggiunge gli \n

```
ZONE = ["NORD", "CENTRO", "SUD"]
```

```
# 1. writelines() da sola: tre elementi incollati.
```

```
with open("zone.txt", "w", encoding="utf-8") as f:
```

```
    f.writelines(ZONE)
```

```
with open("zone.txt", "r", encoding="utf-8") as f:
```

```
    print("writelines nudo:", f.read())
```

```
# 2. Rimedio A: \n dentro ogni elemento, lista costruita con append.
```

```
righe = []
```

```
for zona in ZONE:
```

```
    righe.append(zona + "\n")
```

```
with open("zone.txt", "w", encoding="utf-8") as f:
```

```
    f.writelines(righe)
```

```
with open("zone.txt", "r", encoding="utf-8") as f:
```

```
    print("Rimedio A, a capo:", f.read().count("\n"))
```

```
# writelines nudo: NORDCENTROSUD
```

```
# Rimedio A, a capo: 3
```

6.4 Rimedio B: "\n".join() più \n finale

```
# 3. Rimedio B: .join() del Giorno 02 §10.3, piu' l'a capo finale.
```

```
with open("zone.txt", "w", encoding="utf-8") as f:
```

```
    f.write("\n".join(ZONE) + "\n")
```

```
with open("zone.txt", "r", encoding="utf-8") as f:
```

```
    print("Rimedio B, a capo:", f.read().count("\n"))
```

```
# TRABOCCHETTO: senza il + "\n" finale, join mette gli a capo solo FRA gli
```

```
# elementi: 2 invece di 3, e l'ultima riga resta senza a capo (5.5).
```

```
print("Solo join:", "\n".join(ZONE).count("\n"))
```

```
# Rimedio B, a capo: 3
```

```
# Solo join: 2
```

Stessa lista ZONE della slide precedente. join mette il separatore FRA gli elementi, non dopo l'ultimo: tre zone, due a capo. Il terzo lo aggiungete voi.

6.5 "w" cancella, "a" aggiunge: lo storico degli accessi

```
SERATE = [  
    ["001 Serra", "002 Rinaldi", "003 Motta"],  
    ["004 Ferraro", "005 Nocera"],  
    ["006 Barbato", "007 Guarino"],  
]  
  
def chiudi_serata(percorso, righe, modalita):  
    # Scrive le righe di una serata nel file, con la modalita' indicata.  
    with open(percorso, modalita, encoding="utf-8") as f:  
        for riga in righe:  
            f.write(riga + "\n")  
  
def conta_righe(percorso):  
    # Restituisce il numero di righe del file.  
    with open(percorso, "r", encoding="utf-8") as f:  
        return len(f.readlines())
```

6.5 La ripartenza pulita: un "w" in testa, poi sempre "a"

```
# 1. L'errore dello storico: ogni serata apre in "w" e cancella le altre.

for serata in SERATE:

    chiudi_serata("storico_w.txt", serata, "w")

print("Con w, righe nello storico:", conta_righe("storico_w.txt"))


# 2. RIPARTENZA PULITA: un "w" una volta sola, in testa, per azzerare. Il
#   blocco non scrive niente: pass, come nel Giorno 04 §3.7.

with open("storico_a.txt", "w", encoding="utf-8") as f:

    pass


# 3. Poi ogni serata si accoda in "a". enumerate() del Giorno 07 §6.3.

for numero, serata in enumerate(SERATE, start=1):

    chiudi_serata("storico_a.txt", serata, "a")

    print(f"Con a, dopo la serata {numero}:", conta_righe("storico_a.txt"))


# TRABOCCHETTO: senza il punto 2, una seconda esecuzione del programma
#   accoderebbe alle 7 righe della prima: 10, 12, 14. Due esecuzioni, due
#   file diversi, e un collaudo che non torna mai.


# Con w, righe nello storico: 2

# Con a, dopo la serata 1: 3

# Con a, dopo la serata 2: 5

# Con a, dopo la serata 3: 7
```

6.6 Il buffer: il file sembra vuoto finché non si chiude

```
NOME = "buffer.txt"

with open(NOME, "w", encoding="utf-8") as f:

    f.write("LS123455\n")

    f.write("LS400004\n")

# 1. Il file e' ancora APERTO: lo leggiamo da un secondo open(), come

#     farebbe un altro programma. Il testo sta ancora nel buffer.

with open(NOME, "r", encoding="utf-8") as lettore:

    print("Durante il with:", len(lettore.read()), "caratteri sul disco")

# 2. flush() svuota il buffer sul disco senza chiudere il file.

f.flush()

with open(NOME, "r", encoding="utf-8") as lettore:

    print("Dopo flush():", len(lettore.read()), "caratteri sul disco")


# 3. Uscire dal with chiude il file, e chiudere fa anche il flush.

with open(NOME, "r", encoding="utf-8") as lettore:

    print("Dopo il with:", len(lettore.read()), "caratteri sul disco")


# TRABOCCHETTO: il sintomo compare DOPO la causa e lontano da lei, come il

#     None del Giorno 06 $6.2: il file "vuoto" non ha niente di sbagliato,

#     e' la lettura che arriva troppo presto.


# Durante il with: 0 caratteri sul disco

# Dopo flush(): 18 caratteri sul disco

# Dopo il with: 18 caratteri sul disco
```

Python accumula il testo nel BUFFER e lo porta sul disco tutto insieme, quando è pieno o quando il file si chiude. Il messaggio pratico è uno: fidatevi del with, e leggete un file dopo averlo chiuso. flush() serve solo se un altro programma legge mentre il vostro scrive.

7. I PERCORSI: DOVE SI TROVA IL FILE

Relativo, assoluto, e ancorato allo script

7.1 Directory di lavoro: il file si cerca da dove lanciate

```
from pathlib import Path

# 1. Un nome senza cartelle e' un percorso RELATIVO: si risolve partendo
#     dalla directory di lavoro. Il file nasce li', ovunque sia.
with open("sono_qui.txt", "w", encoding="utf-8") as f:
    f.write("Se leggete questo file, la directory di lavoro è questa.\n")
print("sono_qui.txt esiste:", Path("sono_qui.txt").exists())

# 2. La domanda che conta: directory di lavoro e cartella dello script
#     coincidono? __file__ e .resolve() sono spiegati al 7.4.
cartella_script = Path(__file__).resolve().parent
directory_lavoro = Path.cwd().resolve()
print("Lanciato dalla cartella dello script:", directory_lavoro == cartella_script)

# TRABOCCHETTO: non stampate mai Path.cwd(). E' un percorso assoluto con
#     dentro il nome del vostro utente: cambia da una macchina all'altra, e
#     l'output del programma cambia con lei.

# sono_qui.txt esiste: True
# Lanciato dalla cartella dello script: True
```

7.2 Percorso relativo e assoluto; .. sale di una cartella

Lanciate dalla cartella...	il percorso relativo dei dati è...
giorno_08/	dati/ticket.txt
giorno_08/esercizi/ o giorno_08/soluzioni/	../dati/ticket.txt
giorno_08/esercizi_casa/soluzioni/	../../dati/ticket.txt

Un percorso ASSOLUTO parte dalla radice del disco e vale solo su quel computer. Un percorso RELATIVO parte dalla directory di lavoro e cambia significato con lei. .. vuol dire "sali di una cartella". La tabella ha un difetto: dipende da dove lanciate.

7.2 is_absolute(), resolve() e la cartella di sopra

```
from pathlib import Path

# 1. Relativo o assoluto: is_absolute() risponde "parte dalla radice?".
relativo = Path("dati") / "ticket.txt"
print("Relativo, assoluto?", relativo.is_absolute())
print("Dopo resolve(), assoluto?", relativo.resolve().is_absolute())

# 2. .. sale di una cartella. Prepariamo una sottocartella come quella
# degli esercizi (mkdir e' il paragrafo 8.3).
Path("esercizi").mkdir(exist_ok=True)
giusto = Path("esercizi") / ".." / "dati" / "ticket.txt"
sbagliato = Path("esercizi") / "dati" / "ticket.txt"
print(giusto.as_posix(), "->", giusto.exists())
print(sbagliato.as_posix(), "->", sbagliato.exists())

# TRABOCCHETTO: resolve() rende assoluto il percorso, e un percorso assoluto
# non si stampa (7.1). Si usa per CALCOLARE, mai per MOSTRARE.

# Relativo, assoluto? False
# Dopo resolve(), assoluto? True
# esercizi/../../dati/ticket.txt -> True
# esercizi/dati/ticket.txt -> False
```

7.3 pathlib.Path e l'operatore /: niente stringhe incollate

```
from pathlib import Path

# 1. Il / fra un Path e una stringa costruisce un Path nuovo: non e' una
#     divisione, e' lo stesso simbolo con un altro mestiere.
CARTELLA_DATI = Path("dati")
percorso = CARTELLA_DATI / "spedizioni.txt"

# 2. .as_posix(): sempre la barra dritta, stampa identica ovunque.
print(percorso.as_posix())
print(percorso.name)

# 3. open() accetta un Path cosi' com'e', senza conversioni.
with open(percorso, "r", encoding="utf-8") as f:
    print("Righe:", len(f.readlines()))

# TRABOCCHETTO: la versione "a mano" da' la stessa stringa QUI, su questo
#     sistema. Su Windows no, e non ve ne accorgete finche' il programma non
#     gira sul computer di qualcun altro.
a_mano = "dati" + "/" + "spedizioni.txt"
print("Uguale alla versione a mano:", a_mano == percorso.as_posix())

# dati/spedizioni.txt
# spedizioni.txt
# Righe: 27
# Uguale alla versione a mano: True
```

7.3 Fra due stringhe la / è una divisione: TypeError

```
cartella = "dati"  
percorso = cartella / "ticket.txt"
```

```
Traceback (most recent call last):  
  File "prova.py", line 2, in <module>  
    percorso = cartella / "ticket.txt"  
                ~~~~~^~~~~~  
TypeError: unsupported operand type(s) for /: 'str' and 'str'
```

Perché la / funzioni, almeno uno dei due lati deve essere un Path. TRABOCCHETTO: il messaggio nomina l'operatore, non i percorsi. Quando leggete "unsupported operand type(s) for /", controllate che il primo pezzo sia un Path("..."), non una stringa nuda.

7.4 Path(__file__).resolve().parent: il percorso ancorato

```
import os

from pathlib import Path

# La CONVENZIONE DEL CORSO: costanti in cima al file, in maiuscolo come nel
# Giorno 02 $14.2, calcolate dalla posizione dello SCRIPT.

# Path(__file__) il percorso di questo script
# .resolve() reso assoluto, così .parent funziona sempre
# .parent la cartella che contiene lo script

CARTELLA_SCRIPT = Path(__file__).resolve().parent
CARTELLA_DATI = CARTELLA_SCRIPT / "dati"
CARTELLA_OUTPUT = CARTELLA_SCRIPT / "output"

print("Cartelle:", CARTELLA_DATI.name, CARTELLA_OUTPUT.name)
print("Prima - relativo:", (Path("dati") / "ticket.txt").is_file(),
      "- ancorato:", (CARTELLA_DATI / "ticket.txt").is_file())

# Simuliamo il lancio da un'altra cartella: os.chdir() sposta la directory
# di lavoro, come un cd nel terminale.

os.chdir(CARTELLA_DATI)
print("Dopo cd - relativo:", (Path("dati") / "ticket.txt").is_file(),
      "- ancorato:", (CARTELLA_DATI / "ticket.txt").is_file())

# TRABOCCHETTO: .parent lavora sul TESTO del percorso, non sul disco. Su un
# Path relativo come Path("prova.py") il .parent è ".", cioè di nuovo
# la directory di lavoro. .resolve() prima di .parent lo impedisce.
```

7.4 Un .parent per ogni piano: dove sta lo script

Lo script sta in...	CARTELLA_DATI
accanto a dati/	<code>Path(__file__).resolve().parent / "dati"</code>
esercizi/ o soluzioni/	<code>Path(__file__).resolve().parent.parent / "dati"</code>
esercizi_casa/soluzioni/	<code>Path(__file__).resolve().parent.parent.parent / "dati"</code>

Ogni .parent sale di un piano. Un .parent di troppo o di meno non dà errore lì: lo dà l'open(), con il percorso sbagliato da leggere ad alta voce (3.4). CARTELLA_OUTPUT si costruisce allo stesso modo con "output": i file prodotti vanno lì, mai in dati/.

7.5 Le parti di un percorso: .name, .stem, .suffix, .parent

```
from pathlib import Path

report = Path("output") / "report.finale.txt"

# 1. Le quattro parti. Nessuno split, nessun indice da contare.

print("name  :", report.name)

print("stem  :", report.stem)

print("suffix:", report.suffix)

print("parent:", report.parent.as_posix())

# 2. .with_suffix() costruisce lo stesso percorso con un'altra estensione:

#   utile per salvare il "gemello" .json di un report .txt.

print("json  :", report.with_suffix(".json").name)

# TRABOCCHETTO: lo split a mano da' TRE pezzi, non due, e l'unpacking

#   nome, estensione = ... si fermerebbe con ValueError: too many values

#   to unpack (Giorno 03 $7.4). .suffix guarda solo l'ultimo punto.

print("split :", "report.finale.txt".split("."))

# name  : report.finale.txt

# stem  : report.finale

# suffix: .txt

# parent: output

# json  : report.finale.json

# split : ['report', 'finale', 'txt']
```

Regola di output del corso: si stampa .name, oppure il percorso relativo con .as_posix(); mai un Path intero, mai un percorso assoluto. Su Windows un Path stampato esce con le barre rovesciate: due computer, due output.

7.5 CHICCA: read_text() e write_text(), un file in una riga

```
from pathlib import Path

nota = Path("nota.txt")

# 1. write_text() = open in "w" + write + close. Restituisce i caratteri
#   scritti, come la write() del 6.1.
scritti = nota.write_text("Serra\nRinaldi\n", encoding="utf-8")
print("Caratteri scritti:", scritti)

# 2. read_text() = open in "r" + read + close: una stringa sola.
testo = nota.read_text(encoding="utf-8")
print("Righe:", testo.count("\n"))
print(testo.splitlines())

# TRABOCCHETTO: write_text() apre SEMPRE in "w". Non esiste una versione
#   che accoda: chiamata due volte, la seconda cancella la prima. Per un
#   registro che cresce serve il with con "a" del 6.5.

# Caratteri scritti: 14
# Righe: 2
# ['Serra', 'Rinaldi']
```

Un Path sa leggersi e scriversi da solo: apre, lavora e chiude in una chiamata. Comodo per file piccoli; il with resta la forma da conoscere, l'unica che legge una riga per volta.

7.6 os.path: lo stesso lavoro nel modo vecchio

```
import os

from pathlib import Path

# Lo stesso file nei due mondi: una str costruita da os.path.join, e un
# Path costruito con la barra.

vecchio = os.path.join("dati", "spedizioni.txt")

nuovo = Path("dati") / "spedizioni.txt"

# 1. Stesso percorso: str(nuovo) e' la forma testuale del Path.

print("Stesso percorso:", vecchio == str(nuovo))

# 2. Le stesse domande, fatte nei due modi. Funzione con la stringa dentro
#     contro metodo sul Path: i nomi del Giorno 06 $12.1 in azione.

print("Esiste:", os.path.exists(vecchio), nuovo.exists())

print("È un file:", os.path.isfile(vecchio), nuovo.is_file())

print("Nome:", os.path.basename(vecchio), nuovo.name)

print("Estensione:", os.path.splitext(vecchio)[1], nuovo.suffix)


# TRABOCCHETTO: os.path.join() restituisce una STRINGA. Scrivere
#     vecchio.name si ferma con AttributeError: 'str' object has no
#     attribute 'name'. I metodi di Path su una str non esistono.


# Stesso percorso: True

# Esiste: True True

# È un file: True True

# Nome: spedizioni.txt spedizioni.txt

# Estensione: .txt .txt
```

Lo troverete nel codice degli altri e va riconosciuto; nel corso si scrive pathlib. os.path.dirname(p) è p.parent. In os.path la domanda è una funzione che riceve la stringa, in pathlib è un metodo del percorso.

8. PREVENIRE PRIMA DI APRIRE: exists(), is_file(), mkdir()

Stessa filosofia del Giorno 04, terzo oggetto: il file

8.1 Path.exists() e is_file(): il controllo prima di open()

```
from pathlib import Path

CARTELLA_DATI = Path("dati")

NOMI = ["ticket.txt", "vuoto.txt", "archivio", "ticket_vecchi.txt"]

print(f'{NOME:<19}{exists:<8}is_file')

for nome in NOMI:

    percorso = CARTELLA_DATI / nome

    # 1. exists(): c'e' QUALCOSA con questo nome, file o cartella che sia.

    # 2. is_file(): c'e', ed e' un FILE. E' la domanda giusta prima di open().

    # str() prima della larghezza: vedi il TRABOCCHETTO.

    esiste = str(percorso.exists())

    e_file = str(percorso.is_file())

    print(f'{nome:<19}{esiste:<8}{e_file}')

# TRABOCCHETTO: f"{True:<8}" senza str() non stampa True: stampa 1 seguito
# da sette spazi. Con una larghezza, il bool si formatta come il numero
# che e' sotto (Giorno 02 $8.5). Nessun errore, una colonna di 1 e 0.

# NOME           exists  is_file
# ticket.txt     True    True
# vuoto.txt      True    True
# archivio       True    False
# ticket_vecchi.txt False  False
```

archivio esiste ma è una CARTELLA: aprirla fermerebbe il programma con IsADirectoryError (su Windows PermissionError). Il controllo giusto prima di leggere è is_file(), che copre in un colpo "non esiste" ed "è una cartella".

8.1 Esistere non vuol dire avere dati: il file vuoto

```
from pathlib import Path

VUOTO = Path("dati") / "vuoto.txt"

# 1. Il controllo preventivo passa: il file c'e' ed e' un file.
print("Esiste:", VUOTO.exists(), "- è un file:", VUOTO.is_file())

with open(VUOTO, "r", encoding="utf-8") as f:
    testo = f.read()

# 2. Il secondo controllo, sul CONTENUTO: .strip() == "" copre sia il file
#    di zero caratteri sia quello fatto solo di a capo e spazi.
if testo.strip() == "":
    print("[!]", VUOTO.name, "esiste ma non contiene dati")

# TRABOCCHETTO: su un file vuoto nessuna riga di codice si ferma. Il danno
#    arriva dopo: una media con zero righe al denominatore, cioè' lo
#    ZeroDivisionError del Giorno 07 §9.3.

# Esiste: True - è un file: True
# [!] vuoto.txt esiste ma non contiene dati
```

8.2 carica_righe(): una lista vuota quando il file manca

```
from pathlib import Path

def carica_righe(percorso):
    # Restituisce le righe utili del file, o [] se il file non c'è'.
    # 1. Il ramo che dice di no viene per primo, ed esce subito.
    if not percorso.is_file():
        return []

    righe = []
    with open(percorso, "r", encoding="utf-8") as f:
        for riga in f:
            # 2. La pulizia standard dei paragrafi 5.2 e 5.3.
            riga = riga.rstrip("\n")
            if riga.strip() == "" or riga.startswith("#"):
                continue
            righe.append(riga)

    return righe
```

8.2 Il chiamante decide che cosa fare di []

```
for nome in ["codici_colli.txt", "codici_colli_ieri.txt", "vuoto.txt"]:
    righe = carica_righe(Path("dati") / nome)

    # 3. Il chiamante decide che cosa fare della lista vuota.

    if not righe:

        print(f"{nome:<22} [!] nessuna riga da elaborare")

    else:

        print(f"{nome:<22} {len(righe)} righe, la prima: {righe[0]}")

# TRABOCCHETTO: [] risponde nello stesso modo a "il file non c'e'" e a "il
#   file c'e' ma e' vuoto". Se al chiamante serve distinguere i due casi,
#   la funzione deve restituire DUE valori, lista ed esito (Giorno 06 §10.1).

# codici_colli.txt          13 righe, la prima: LS123455
# codici_colli_ieri.txt     [!] nessuna riga da elaborare
# vuoto.txt                 [!] nessuna riga da elaborare
```

8.3 Scrivere in una cartella che non c'è: FileNotFoundError

```
from pathlib import Path

CARTELLA_OUTPUT = Path("output")
destinazione = CARTELLA_OUTPUT / "report.txt"
with open(destinazione, "w", encoding="utf-8") as f:
    f.write("Report di prova\n")

Traceback (most recent call last):
  File "prova.py", line 5, in <module>
    with open(destinazione, "w", encoding="utf-8") as f:
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: 'output/report.txt'
```

8.3 mkdir(parents=True, exist_ok=True): output/ si crea da sola

```
from pathlib import Path

CARTELLA_OUTPUT = Path("output")
BACKUP = CARTELLA_OUTPUT / "backup"

# 1. Due esecuzioni simulate con un for: con exist_ok=True la seconda non
# protesta. parents=True crea output/ e output/backup/ insieme.
for esecuzione in range(1, 3):

    BACKUP.mkdir(parents=True, exist_ok=True)

    print(f"Esecuzione {esecuzione}: backup pronta?", BACKUP.is_dir())

# 2. Adesso la scrittura del blocco precedente funziona.
destinazione = CARTELLA_OUTPUT / "report.txt"

with open(destinazione, "w", encoding="utf-8") as f:

    f.write("Report di prova\n")

print("Scritto:", destinazione.as_posix())

# TRABOCCHETTO: senza parents=True, BACKUP.mkdir() su una output/ che non
# esiste ancora si ferma con FileNotFoundError: la cartella di mezzo
# manca. Sono due protezioni diverse, e servono tutte e due.

# Esecuzione 1: backup pronta? True
# Esecuzione 2: backup pronta? True

# Scritto: output/report.txt
```

8.4 x con exists(): consegna_report() decide invece di fermarsi

```
from pathlib import Path

CARTELLA_OUTPUT = Path("output")

CARTELLA_OUTPUT.mkdir(parents=True, exist_ok=True)

REPORT = CARTELLA_OUTPUT / "chiusura_mese.txt"


def consegna_report(percorso, testo):
    # Scrive il report solo se non esiste già'. Restituisce True se scritto.
    # Il ramo che rifiuta viene per primo, come nel Giorno 04 §4.4.
    if percorso.exists():
        return False

    with open(percorso, "w", encoding="utf-8") as f:
        f.write(testo)

    return True
```

8.4 Tre tentativi, un report solo

```
for tentativo in range(1, 4):
    if consegna_report(REPORT, f"Versione {tentativo}\n"):
        print(f"Tentativo {tentativo}: [OK] report consegnato")
    else:
        print(f"Tentativo {tentativo}: [!] report già consegnato, non lo tocco")

with open(REPORT, "r", encoding="utf-8") as f:
    print("Nel file:", f.read(), end="")

# TRABOCCHETTO: la funzione restituisce un bool e non stampa niente: e'
#   chi la chiama a decidere il messaggio (Giorno 06 $5.2). Con un print
#   dentro la funzione, il ramo else del chiamante non saprebbe che dire.

# Tentativo 1: [OK] report consegnato
# Tentativo 2: [!] report già consegnato, non lo tocco
# Tentativo 3: [!] report già consegnato, non lo tocco
# Nel file: Versione 1
```

8.5 Il limite del controllo: il file che compare dopo exists()

```
from pathlib import Path

REPORT = Path("report_turno.txt")

# 1. Il controllo: il nome e' libero, quindi decidiamo di creare il file.
libero = not REPORT.exists()
print("Controllo: il nome è libero?", libero)

# 2. Fra il controllo e l'apertura un altro programma crea lo stesso file.
#    Lo simuliamo con due righe: per il nostro codice, e' tempo che passa.
with open(REPORT, "w", encoding="utf-8") as altro:
    altro.write("Report del collega\n")

# 3. L'apertura si fida di un controllo ormai vecchio.
if libero:
    with open(REPORT, "x", encoding="utf-8") as f:
        f.write("Il nostro report\n")

# Commenti in coda, per non spostare i numeri di riga del traceback.
# TRABOCCHETTO: con "w" al posto di "x" al punto 3 non ci sarebbe nessun
# errore, e il report del collega sarebbe sparito. Qui almeno ci si
# ferma: e' "x" l'unica difesa che non dipende dal momento del controllo.
```

8.5 Il controllo era vero alla riga 6, falso alla riga 16

```
Controllo: il nome è libero? True
Traceback (most recent call last):
  File "prova.py", line 16, in <module>
    with open(REPORT, "x", encoding="utf-8") as f:
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
FileExistsError: [Errno 17] File exists: 'report_turno.txt'
```

Nessun if può chiudere quel varco, perché l'if stesso è un momento separato dall'apertura. Esiste uno strumento che raccoglie l'errore dopo che è successo: ci arriveremo quando arriveremo alle eccezioni. Il controllo preventivo resta la prima linea di difesa.

8.5 Che cosa il controllo preventivo non vede

Che cosa succede	Che cosa vedreste	Un if lo previene?
il file sparisce fra exists() e open()	FileNotFoundError	no: il controllo è già passato
il file c'è ma non avete i permessi per leggerlo	PermissionError	solo in parte
il file è aperto e bloccato da un altro programma (Windows)	PermissionError	no
il percorso è una cartella (su Windows)	PermissionError	sì, con is_file()
il contenuto è in un'altra codifica (2.4)	UnicodeDecodeError	no: lo scopre solo la lettura

9. RIGHE CON PIÙ CAMPI: IL FILE DELIMITATO

Record, campi, separatore: e la riga storta che arriva sempre

9.1 Una riga, un record: .split(";") e l'unpacking

```
from pathlib import Path

PERCORSO = Path("dati") / "spedizioni.txt"

# 1. Tutte le righe in una lista (§4.2), per prendere le prime tre righe di
#     dati con una fetta, come nel Giorno 07 §7.1.
with open(PERCORSO, "r", encoding="utf-8") as f:
    righe = f.readlines()

# 2. righe[0] e' il commento di testa: la fetta [1:4] da' tre RECORD.
for riga in righe[1:4]:
    riga = riga.rstrip("\n")

    # 3. L'unpacking del Giorno 03: tre pezzi a destra, tre nomi a sinistra.
    codice, zona, peso = riga.split(";")

    # 4. Dal file arriva SEMPRE testo: il peso diventa un numero solo qui.
    peso = float(peso)

    print(f"{codice:<17}{zona:<8}{peso:>6.1f} kg")

# TRABOCCHETTO: senza rstrip("\n") questo programma gira lo stesso, perche'
#     float("12.5\n") perdona l'a capo. Lo paga un campo di TESTO in fondo alla
#     riga: una zona che valesse "NORD\n" non sarebbe mai uguale a "NORD".
print("Tipo del peso dopo float():", type(peso))
```

9.1 CHICCA: split(";", 2), il terzo campo resta intero

```
riga = "TK031;RECLAMO;Attesa di due ore; nessuno ha avvisato"

# Senza limite: quattro pezzi, e un unpacking a tre nomi si fermerebbe.
print(len(riga.split(";")))

# Con limite 2: due tagli, quindi TRE pezzi. Il numero conta i tagli.
id_ticket, categoria, testo = riga.split(";", 2)
print(id_ticket)
print(categoria)
print(testo)

# TRABOCCHETTO: split(";", 3) non protegge di piu', protegge di meno: tre
#   tagli, quattro pezzi, e l'unpacking torna a fermarsi con ValueError.

# 4

# TK031

# RECLAMO

# Attesa di due ore; nessuno ha avvisato
```

9.2 La riga con un campo in meno ferma tutto: ValueError

```
riga = "LS-2026-TO-0990;NORD"  
codice, zona, peso = riga.split(";")
```

```
Traceback (most recent call last):  
  File "prova.py", line 2, in <module>  
    codice, zona, peso = riga.split(";")  
ValueError: not enough values to unpack (expected 3, got 2)
```

Alla riga 8 del file il magazziniere ha dimenticato il peso. Due pezzi a destra, tre nomi a sinistra. TRABOCCHETTO: su un file si ferma A METÀ lettura, e le righe prima sono già state elaborate: se stava scrivendo un report, il report esce mezzo fatto. Da qui si omettono le righe ^^.

9.2 Prima si divide e si conta, poi si spacchetta

```
from pathlib import Path

PERCORSO = Path("dati") / "spedizioni.txt"

# Il numero di campi e' il contratto del file: scritto una volta, in cima.
CAMPI_ATTESI = 3

buone = 0

with open(PERCORSO, "r", encoding="utf-8") as f:

    # enumerate(f, 1) numera da 1 come l'editor (Giorno 07 $6.3): il numero
    # stampato e' quello che cercherete aprendo il file.

    for numero, riga in enumerate(f, 1):

        riga = riga.rstrip("\n")

        if riga.strip() == "" or riga.startswith("#"):

            continue

        campi = riga.split(";")

        if len(campi) != CAMPI_ATTESI:

            print(f"[!] riga {numero}: {len(campi)} campi invece di "

                  f"{CAMPI_ATTESI} -> {riga}")

            continue

        # Qui i campi sono tre per costruzione: l'unpacking non puo' fallire.
        codice, zona, peso = campi

        buone += 1

# TRABOCCHETTO: rifare riga.split(";") per l'unpacking invece di riusare
# campi funziona, ma divide due volte: il giorno che controllo e unpacking
# usano separatori diversi, divergono in silenzio.

print(f"Righe con {CAMPI_ATTESI} campi: {buone}")
```

9.3 Il campo numerico: float("dodici")

```
peso = "dodici"  
valore = float(peso)
```

```
Traceback (most recent call last):  
  File "prova.py", line 2, in <module>  
    valore = float(peso)  
ValueError: could not convert string to float: 'dodici'
```

Alla riga 17 del file il peso è scritto in lettere, alla 13 con la virgola italiana. float() accetta solo la scrittura di un numero. Il messaggio ripete fra apici il testo rifiutato: leggetelo, spesso c'è uno spazio che non vedevate.

9.3 e_numero(): controllare prima di float()

```
def e_numero(testo):  
    # Vero se il testo, ripulito, e' un numero decimale senza segno.  
  
    # 1. Pulizia: spazi ai bordi via, virgola italiana -> punto (Giorno 03).  
    pulito = testo.strip().replace(",", ".")  
  
    # 2. Si toglie UN punto solo, il separatore decimale: il terzo argomento  
    #    di replace dice "al massimo una sostituzione".  
  
    return pulito.replace(".", "", 1).isdecimal()  
  
PROVE = ["12.5", "7,3", " 5.0 ", "dodici", "", "12.500,00", "-3.5", "1e3"]  
  
for prova in PROVE:  
    # Le quadre rendono visibili gli spazi e la stringa vuota.  
    etichetta = "[" + prova + "]"  
  
    print(f"{etichetta:<13}-> {e_numero(prova)}")  
  
# TRABOCCHETTO: senza quell'1, "12.500,00" diventa "12.500.00", perde TUTTI  
# i punti, passa il controllo come "1250000" e poi float() si ferma con  
# ValueError: could not convert string to float: '12.500.00'.  
# TRABOCCHETTO: si converte la stringa PULITA, non l'originale: float("7,3")  
# si ferma con ValueError: could not convert string to float: '7,3'.
```

9.3 Otto prove, e le ultime tre lette con onestà

```
[12.5]      -> True
[7,3]       -> True
[ 5.0 ]     -> True
[dodici]    -> False
[]          -> False
[12.500,00] -> False
[-3.5]      -> False
[1e3]       -> False
```

12.500,00 è ambiguo, e rifiutarlo è giusto. Ma -3.5 e 1e3 sono numeri validi che float() convertirebbe. Per un peso non è grave, per un saldo sì: qui il controllo preventivo costa più di quanto protegge. La strada che raccoglie il rifiuto dopo arriverà con le eccezioni.

9.4 motivo_scarto(): il motivo di ogni riga rifiutata

```
from pathlib import Path

PERCORSO = Path("dati") / "spedizioni.txt"

CAMPI_ATTESI = 3

ZONE_VALIDE = ["NORD", "CENTRO", "SUD", "ISOLE"]

def e_numero(testo):

    pulito = testo.strip().replace(",", ".")

    return pulito.replace(".", "", 1).isdecimal()

def motivo_scarto(campi):

    # Restituisce "" se il record e' buono, altrimenti il PRIMO motivo.

    # Dal controllo piu' grossolano al piu' fine, come in 05.9. Il return

    # anticipato del Giorno 06 $5.3 esce al primo difetto trovato.

    if len(campi) != CAMPI_ATTESI:

        return "numero di campi"

    # Da qui in poi i campi sono tre: l'unpacking non puo' fallire.

    codice, zona, peso = campi

    if not e_numero(peso):

        return "peso non numerico"

    if zona.strip().upper() not in ZONE_VALIDE:

        return "zona sconosciuta"

    return ""
```

9.4 Scartare e contare: il conta-occorrenze sui motivi

```
buone = 0

motivi = {}

with open(PERCORSO, "r", encoding="utf-8") as f:

    for riga in f:

        riga = riga.rstrip("\n")

        if riga.strip() == "" or riga.startswith("#"):

            continue

        motivo = motivo_scarto(riga.split(";"))

        if motivo == "":

            buone += 1

        else:

            # Il conta-occorrenze del Giorno 07 $12.2, questa volta sui motivi.

            motivi[motivo] = motivi.get(motivo, 0) + 1

            print(f"[SCARTO] {motivo:<18} {riga}")

# TRABOCCHETTO: controllate la zona PRIMA del numero di campi, e sulla riga

# LS-2026-TO-0990;NORD l'unpacking si ferma con ValueError: not enough

# values to unpack (expected 3, got 2). Ogni controllo prepara il dopo.

print("Righe buone:", buone)

for motivo, quante in motivi.items():

    print(f" {motivo:<18} {quante}")
```

9.4 Tre scarti, ventidue righe buone

```
[SCARTO] numero di campi      LS-2026-TO-0990;NORD
[SCARTO] peso non numerico    LS-2026-BA-1001;SUD;dodici
[SCARTO] zona sconosciuta     LS-2026-GE-1002;ESTERO;3.0
Righe buone: 22
```

```
numero di campi      1
peso non numerico    1
zona sconosciuta     1
```

Guardate le righe che NON sono fra gli scarti. La 13 ha il peso 7,3, e `e_numero()` la accetta. La 21 ha spazi ovunque e la zona in minuscolo, e `.strip().upper()` la rende buona. Pulire dopo lo split (Giorno 03 §7.5) fa la differenza fra 20 righe buone e 22.

9.5 Il file con intestazione: saltarla senza perderla

```
from pathlib import Path

PERCORSO = Path("dati") / "ordini.csv"

with open(PERCORSO, "r", encoding="utf-8") as f:

    # 1. readline() legge UNA riga e sposta avanti il segno di lettura: il
    #     for qui sotto non la vedrà mai.

    colonne = f.readline().rstrip("\n").split(";")

    # 2. I nomi arrivano dal FILE: se un giorno cambiano, cambia il report.

    print(f"{colonne[0].upper():<9}{colonne[1].upper():<13}"

          f"{colonne[3].upper():>8}")

    righe_dati = 0

    for riga in f:

        if riga.strip() == "":

            continue

        righe_dati += 1

        # 3. Le prime tre righe di dati, per vedere la forma.

        if righe_dati <= 3:

            campi = riga.rstrip("\n").split(";")

            print(f"{campi[0]:<9}{campi[1]:<13}{campi[3]:>8}")

# TRABOCCHETTO: campi[3] è ancora TESTO, "99.80". Allineato con :>8 sembra

#     un numero; con :>8.2f si ferma con

#     ValueError: Unknown format code 'f' for object of type 'str'.

print("Colonne:", len(colonne), "- righe di dati:", righe_dati)
```

9.5 I nomi delle colonne arrivano dal file

```
CODICE    CATEGORIA    IMPORTO
NS-1001   AUDIO        99.80
NS-1002   INFORMATICA   89.00
NS-1003   ACCESSORI     61.50
Colonne: 5 - righe di dati: 15
```

Quindici righe di dati, ma nel Giorno 07 gli ordini erano dodici. Le tre in più sono righe rotte, e il capitolo 10 le scarta.

9.6 Il modulo csv: le stesse righe, con le virgolette gestite

```
import csv

from pathlib import Path

PERCORSO = Path("reclami.csv")

# Due righe: la prima ha il testo fra virgolette, con un ";" che non separa.
with open(PERCORSO, "w", encoding="utf-8") as f:

    f.write('TK031;RECLAMO;"Attesa di due ore; nessuno ha avvisato"\n')

    f.write("TK032;ALTRO;Richiesta di informazioni\n")


# 1. Lo split di 9.1 non sa niente di virgolette: taglia a ogni ";".
with open(PERCORSO, "r", encoding="utf-8") as f:

    for riga in f:

        print(len(riga.rstrip("\n").split(";")), "campi con split")


# 2. csv.reader consegna a ogni giro una LISTA di campi già divisa.
#     newline="" e' la raccomandazione del modulo: gli a capo li gestisce lui.
with open(PERCORSO, "r", encoding="utf-8", newline="") as f:

    for campi in csv.reader(f, delimiter=";"):

        print(len(campi), "campi con csv:", campi[2])


# TRABOCCHETTO: senza delimiter=";" il modulo usa la virgola: ogni riga e'
# UN campo solo, e campi[2] si ferma con IndexError: list index out of range.
```

10. DAL FILE ALLA STRUTTURA DATI

La promessa del Giorno 07: cambiano solo le prime righe

10.1 carica_spedizioni(): dal file a una lista di tuple

```
def carica_spedizioni(percorso):  
    spedizioni = []  
    scartate = 0  
    with open(percorso, "r", encoding="utf-8") as f:  
        for riga in f:  
            riga = riga.rstrip("\n")  
            if riga.strip() == "" or riga.startswith("#"):  
                continue  
            campi = riga.split(";")  
            # 2. Il cortocircuito del Giorno 04: se i campi non sono tre,  
            #     campi[2] non viene nemmeno guardato.  
            if len(campi) != CAMPI_ATTESI or not e_numero(campi[2]):  
                scartate += 1  
                continue  
            codice = campi[0].strip()  
            zona = campi[1].strip().upper()  
            if zona not in ZONE_VALIDE:  
                scartate += 1  
                continue  
            # 3. Si converte la stringa PULITA, la stessa che si e' controllata.  
            peso = float(campi[2].strip().replace(",", "."))  
            spedizioni.append((codice, zona, peso))  
        # 4. Due valori consegnati insieme: il return multiplo del Giorno 06 $10.1.  
    return spedizioni, scartate
```

10.1 Le tuple che arrivano dal file

```
spedizioni, scartate = carica_spedizioni(PERCORSO)

print("Spedizioni caricate:", len(spedizioni))

print("Righe scartate:", scartate)

print("Prima riga:", spedizioni[0])

print("Riga 21 ripulita:", spedizioni[16])

print("Riga 13 con virgola:", spedizioni[10])


# TRABOCCHETTO: il return rientrato di un livello, dentro il for, e' il
#   return nel ciclo del Giorno 06 E7: la funzione esce alla prima riga
#   buona. "Spedizioni caricate: 1", e spedizioni[16] si ferma con
#   IndexError: list index out of range.


# Spedizioni caricate: 22
# Righe scartate: 3
# Prima riga: ('LS-2026-MI-0042', 'NORD', 12.5)
# Riga 21 ripulita: ('LS-2026-MI-0999', 'NORD', 5.0)
# Riga 13 con virgola: ('LS-2026-NA-0977', 'SUD', 7.3)
```

10.2 Il cruscotto del Giorno 07: carica_ordini() dal file

```
PERCORSO = Path("dati") / "ordini.csv"

CAMPI_ATTESI = 5

LARGHEZZA_BARRA = 40

def carica_ordini(percorso):

    ordini = []

    scartati = 0

    with open(percorso, "r", encoding="utf-8") as f:

        # L'intestazione si consuma prima del for ($9.5).

        f.readline()

        for riga in f:

            riga = riga.rstrip("\n")

            if riga.strip() == "":

                continue

            campi = riga.split(";")

            if len(campi) != CAMPI_ATTESI:

                scartati += 1

                continue

            codice, categoria, mese, importo, pezzi = campi

            if not e_numero(importo) or not pezzi.isdecimal():

                scartati += 1

                continue

            # La tupla ha la STESSA forma di quelle scritte a mano in 07.8.

            ordini.append((codice, categoria, mese, float(importo), int(pezzi)))

    return ordini, scartati
```

10.2 L'unica riga nuova rispetto a 07.8

```
# --- aggrega_per_categoria() e riepilogo(): copiate identiche da 07.8 ---

# L'UNICA riga nuova rispetto a 07.8: la lista arriva dal file.
ordini, scartati = carica_ordini(PERCORSO)
fatturato, pezzi_venduti = riepilogo(ordini)

print(f"Ordini ..... {len(ordini)}")
print(f"Pezzi venduti ..... {pezzi_venduti}")
print(f"Fatturato ..... {fatturato:>7.2f} euro")
print(f"Righe scartate ..... {scartati}")

importi_cat, pezzi_cat, quanti_cat = aggrega_per_categoria(ordini)
print("PER CATEGORIA")

for categoria, importo in importi_cat.items():
    quota = importo / fatturato
    barra = "#" * int(quota * LARGHEZZA_BARRA)
    print(f"{categoria:<13}{importo:>8.2f} euro  "
          f"{pezzi_cat[categoria]:>2} pezzi   {quota:>5.1%}   {barra}")

# TRABOCCHETTO: togliete.rstrip("\n") da carica_ordini. L'ultimo campo vale
# "2\n", "2\n".isdecimal() e' False, e TUTTE le righe finiscono negli
# scarti: "Ordini ..... 0", "Righe scartate ..... 15", nessun errore.
```

10.2 Lo stesso output di 07.8, con i dati da file

```
Ordini ..... 12
Pezzi venduti ..... 21
Fatturato ..... 1457.60 euro
Righe scartate ..... 3
PER CATEGORIA
AUDIO          299.40 euro    6 pezzi    20.5%    #####
INFORMATICA    383.00 euro    5 pezzi    26.3%    #####
ACCESSORI      157.50 euro    5 pezzi    10.8%    ###
VIDEO          617.70 euro    5 pezzi    42.4%    #####
```

12 ordini, 21 pezzi, 1457.60 euro, VIDEO al 42.4%, le barre lunghe uguali. Anche l'ORDINE delle categorie è lo stesso: le tre righe rifiutate non arrivano mai a .get() e non creano chiavi. Il lavoro del Giorno 07 non si butta: si alimenta.

10.3 Dal file a un dizionario: il listino zona -> prezzo

```
from pathlib import Path

PERCORSO_LISTINO = Path("dati") / "listino_zone.txt"

SPEDIZIONI = [("LS-2026-MI-0042", "NORD", 12.5),
               ("LS-2026-NA-0913", "SUD", 6.7),
               ("LS-2026-GE-1002", "ESTERO", 3.0),
               ("LS-2026-PA-1571", "ISOLE", 8.5)]

def carica_listino(percorso):
    # Restituisce il dizionario zona -> prezzo al collo letto dal file.
    listino = {}
    with open(percorso, "r", encoding="utf-8") as f:
        for riga in f:
            riga = riga.rstrip("\n")
            if riga.strip() == "" or riga.startswith("#"):
                continue
            zona, prezzo = riga.split(";")
            # La chiave si normalizza QUI, quando entra: chi cerca dopo puo'
            # fidarsi che le chiavi siano tutte maiuscole e senza spazi.
            listino[zona.strip().upper()] = float(prezzo)
    return listino
```

10.3 in prima dell'accesso: l'ESTERO fuori listino

```
listino = carica_listino(PERCORSO_LISTINO)

print(listino)

totale = 0

a_listino = 0

for codice, zona, peso in SPEDIZIONI:

    # Il controllo con in PRIMA dell'accesso (Giorno 07 $10.3): l'ESTERO non

    # e' un errore dei dati, e' una zona da preventivare a mano.

    if zona not in listino:

        print(f"{codice:<17}{zona:<8}[!] fuori listino")

        continue

    prezzo = listino[zona]

    totale += prezzo

    a_listino += 1

    print(f"{codice:<17}{zona:<8}{prezzo:>6.2f}")

print(f"Totale a listino: {totale:.2f} euro ")

    f"({a_listino} colli su {len(SPEDIZIONI)})")

# TRABOCCHETTO: senza l'if, listino["ESTERO"] si ferma con KeyError: 'ESTERO'

# alla terza spedizione, e le prime due sono gia' stampate.

# LS-2026-MI-0042  NORD      8.50

# LS-2026-NA-0913  SUD      9.50

# LS-2026-GE-1002  ESTERO   [!] fuori listino

# LS-2026-PA-1571  ISOLE    14.00
```

10.3 Il float() dimenticato: la stringa ripetuta cinque volte

```
# Un prezzo come arriva dal file, SENZA float(): e' ancora testo.
prezzo = "8.50"
colli = 5

# Stringa per intero: Python non protesta, RIPETE la stringa cinque volte.
# E' la moltiplicazione di stringhe del Giorno 02, qui nel posto sbagliato.
print(prezzo * colli)

# La stessa riga con la conversione: il numero che volevate.
print(float(prezzo) * colli)

# TRABOCCHETTO: il sintomo arriva lontano dalla causa, come il None del
#   Giorno 06 §6.2. Lo vedrete in un report, non nella riga sbagliata.

# 8.508.508.508.508.50
# 42.5
```

10.4 Il conta-occorrenze su righe che arrivano dal disco

```
from pathlib import Path

PERCORSO = Path("dati") / "ticket.txt"

conteggi = {}

with open(PERCORSO, "r", encoding="utf-8") as f:

    for riga in f:

        riga = riga.rstrip("\n")

        if riga.strip() == "" or riga.startswith("#"):

            continue

        # La CHICCA di 9.1: il testo libero resta un pezzo solo.

        id_ticket, categoria, testo = riga.split(";", 2)

        # La riga del Giorno 07 $12.2, identica: cambia solo da dove arriva

        # la categoria.

        conteggi[categoria] = conteggi.get(categoria, 0) + 1

# Il denominatore e' il numero di TICKET, non di categorie (Giorno 07 $12.3).

totale = sum(conteggi.values())

for categoria, quanti in conteggi.items():

    quota = quanti / totale

    print(f" {categoria:<13}{quanti:>3} {quota:>6.1%} {'#' * quanti}")

print(f" {'TOTALE':<13}{totale:>3}")

# TRABOCCHETTO: con split(";",) e tre nomi, un paziente che scrive un ";" nel

# testo ferma la lettura con ValueError: too many values to unpack. Oggi

# nessuno l'ha fatto: e' il genere di difetto che esplode fra sei mesi.
```

10.4 Trenta ticket, cinque categorie

PRENOTAZIONE	7	23.3%	#####
REFERTO	7	23.3%	#####
PAGAMENTO	6	20.0%	#####
RECLAMO	6	20.0%	#####
ALTRO	4	13.3%	####
TOTALE	30		

Le percentuali sommano a 99.9, non a 100: ognuna è arrotondata a un decimale, e gli arrotondamenti non si compensano. Non è un errore del programma, ed è bene saperlo prima che qualcuno in riunione sommi la colonna.

10.4 CHICCA: max(conteggi, key=conteggi.get)

```
conteggi = {"PRENOTAZIONE": 7, "REFERTO": 7, "PAGAMENTO": 6,
            "RECLAMO": 6, "ALTRO": 4}

# max() su .values() risponde QUANTO, come nel Giorno 07 §9.2.
print(max(conteggi.values()))

# key=conteggi.get: max misura ogni chiave con il suo VALORE, e restituisce
# la chiave. E' max a chiamare get, una chiave per volta: voi la consegnate.
print(max(conteggi, key=conteggi.get))

# TRABOCCHETTO: a parita' vince la PRIMA chiave incontrata: PRENOTAZIONE e
#   REFERTO hanno 7, e risponde PRENOTAZIONE perche' e' comparsa prima.
# TRABOCCHETTO: max(conteggi) senza key confronta i NOMI in ordine
#   alfabetico, e qui risponde REFERTO. Giusto per caso, a pari merito.

# 7

# PRENOTAZIONE
```

10.5 Il file di configurazione CHIAVE=valore

```
def carica_configurazione(percorso):  
    soglia = 0  
    indizi = []  
    with open(percorso, "r", encoding="utf-8") as f:  
        for riga in f:  
            riga = riga.rstrip("\n")  
            if riga.strip() == "" or riga.startswith("#"):  
                continue  
            # 1. Un taglio solo, sul primo "=": il valore puo' contenerne altri.  
            chiave, valore = riga.split("=", 1)  
            # 2. Pulizia su ENTRAMBI i pezzi: nel file c'e' " INDIZIO = urgente;1".  
            chiave = chiave.strip().upper()  
            valore = valore.strip()  
            if chiave == "SOGLIA":  
                soglia = int(valore)  
            elif chiave == "INDIZIO":  
                parola, peso = valore.split(";")  
                indizi.append((parola.strip().lower(), int(peso)))  
            else:  
                print(f"[!] chiave sconosciuta ignorata: {chiave}")  
    # TRABOCCHETTO: senza .strip() sulla chiave, " INDIZIO " non e' "INDIZIO":  
    # l'indizio urgente finisce fra le chiavi sconosciute, gli indizi  
    # scendono a 6, e il filtro perde un punto senza dire niente.  
    return soglia, indizi
```

10.5 Cambiare la soglia senza toccare il codice

```
ORIGINALE = Path("dati") / "configurazione_filtro.txt"
PROVA = Path("configurazione_prova.txt")

TESTO = "Pratica di rimborso urgente: aggiorni le credenziali del portale"

soglia, indizi = carica_configurazione(ORIGINALE)
print("Soglia:", soglia, "- indizi:", len(indizi))

for parola, peso in indizi:
    print(f"    {parola:<20}{peso}")

# La direzione alza la soglia: si cambia un FILE, non il programma.
testo_config = ORIGINALE.read_text(encoding="utf-8")
PROVA.write_text(testo_config.replace("SOGLIA=5", "SOGLIA=8"), encoding="utf-8")

for percorso in [ORIGINALE, PROVA]:
    soglia, indizi = carica_configurazione(percorso)
    punti = punteggio_spam(TESTO, indizi)

    # Stessa regola di 07.7: dalla soglia in su si blocca.
    if punti >= soglia:
        verdetto = "SPAM"
    else:
        verdetto = "LEGITTIMO"

    print(f"{percorso.name:<27} soglia {soglia}: {punti} punti -> {verdetto}")

# TRABOCCHETTO: la configurazione si carica UNA volta, all'avvio. Chi la
#   ricaricasse dentro il ciclo dei messaggi rileggerebbe il file a ogni
#   messaggio: stesso risultato, e quindici letture al posto di una.
```

10.5 Stesso messaggio, stessi punti, verdetto opposto

```
[!] chiave sconosciuta ignorata: LINGUA
Soglia: 5 - indizi: 7
  gratis          3
  vinci           3
  clicca subito   4
  offerta imperdibile 2
  urgente         1
  credenziali     4
  rimborso       2
[!] chiave sconosciuta ignorata: LINGUA
configurazione_filtro.txt  soglia 5: 7 punti -> SPAM
[!] chiave sconosciuta ignorata: LINGUA
configurazione_prova.txt   soglia 8: 7 punti -> LEGITTIMO
```

Sette punti: rimborso 2, urgente 1, credenziali 4. Nel blocco etichettato di 07.7 quel messaggio era un falso allarme: la direzione che alza la soglia lo fa passare senza che nessuno riscriva una riga. Se è una buona idea lo dice l'esercizio 08.8.

11. SCRIVERE UN REPORT E UN REGISTRO

Il report da allegare, il registro di quello che si è fatto, gli scarti

11.1 Comporre prima, scrivere dopo: le righe del report

```
TOTALI = [("NORD", 5, 56.1), ("CENTRO", 5, 90.7),
          ("SUD", 7, 141.1), ("ISOLE", 5, 78.6)]

def componi_report(totali):

    # 1. Il report e' una LISTA DI STRINGHE: nessun print qui dentro.

    righe = [f"{'ZONA':<8}{'COLLI':>6}{'KG':>8}"]

    colli_tot = 0

    kg_tot = 0

    for zona, colli, kg in totali:

        # 2. Stesse larghezze dell'intestazione (Giorno 03 cap. 9).

        righe.append(f"{'zona':<8}{'colli':>6}{'kg':>8.1f}")

        colli_tot += colli

        kg_tot += kg

    righe.append(f"{'TOTALE':<8}{'colli_tot':>6}{'kg_tot':>8.1f}")

    return righe


# 3. Chi chiama ha in mano le righe, e ci fa quello che vuole.

righe = componi_report(TOTALI)

print("Righe composte:", len(righe))

for riga in righe:

    print(riga)


# TRABOCCHETTO: con print al posto di append, la funzione stampa le righe e
# restituisce la sola intestazione. A video tutto sembra giusto; il file
# che scriverete con quella lista ha una riga sola (Giorno 06 $5.1).
```

11.2 Lo stesso report a video e su file con un parametro solo

```
from pathlib import Path

REPORT = Path("report_zone.txt")

RIGHE = ["ZONA      COLLI", "NORD      5", "SUD      7"]

def stampa_report(righe, destinazione=None):
    # Nessun if: print con file=None scrive sullo schermo (CHICCA qui sotto).
    for riga in righe:
        print(riga, file=destinazione)

# 1. Senza destinazione: a video.
stampa_report(RIGHE)

# 2. Con destinazione: il file APERTO, non il suo nome. La funzione non apre
#    e non chiude niente: il with resta a chi chiama.
with open(REPORT, "w", encoding="utf-8") as f:
    stampa_report(RIGHE, destinazione=f)

# TRABOCCHETTO: passare destinazione=REPORT, cioe' il percorso, si ferma con
#    AttributeError: 'PosixPath' object has no attribute 'write' (su Windows
#    'WindowsPath'). print vuole un file aperto, non un indirizzo.
print("---", REPORT.name, "---")

print(REPORT.read_text(encoding="utf-8"), end="")
```

11.2 CHICCA: print("testo", file=None) scrive sullo schermo

```
# Il default di file, dentro print, e' None: le due righe vanno a video.
```

```
print("Prima riga, file=None scritto esplicitamente", file=None)
```

```
# Lo stesso None passato per variabile: e' il caso di stampa_report().
```

```
destinazione = None
```

```
print("Seconda riga, None arrivato da una variabile", file=destinazione)
```

```
# TRABOCCHETTO: file="" non e' None. Una stringa vuota non e' un file:
```

```
#   AttributeError: 'str' object has no attribute 'write'.
```

```
# Prima riga, file=None scritto esplicitamente
```

```
# Seconda riga, None arrivato da una variabile
```

11.3 Il registro in modalità a: una riga per evento

```
from pathlib import Path

REGISTRO = Path("cruscotto.log")

def registra(percorso, numero, livello, messaggio):

    # :03d scrive l'intero su tre cifre con gli zeri davanti: 1 -> 001.

    # Le righe restano allineate e si ordinano bene anche come testo.

    with open(percorso, "a", encoding="utf-8") as f:

        f.write(f"{numero:03d} | {livello:<6} | {messaggio}\n")

def esegui(riparti_pulito):

    if riparti_pulito:

        # Aprire in "w" e non scrivere niente SVUOTA il file: e' la

        # ripartenza pulita di $6.5.

        with open(REGISTRO, "w", encoding="utf-8") as f:

            f.write("")

        registra(REGISTRO, 1, "INFO", "avvio: lettura di ordini.csv")

        registra(REGISTRO, 2, "AVVISO", "3 righe scartate")

        registra(REGISTRO, 3, "INFO", "fine: report scritto")

def conta_righe(percorso):

    with open(percorso, "r", encoding="utf-8") as f:

        return len(f.readlines())
```

11.3 Due esecuzioni, lo stesso file: la ripartenza pulita

```
esegui(True)

print("Prima esecuzione:", conta_righe(REGISTRO), "righe")

esegui(True)

print("Seconda esecuzione:", conta_righe(REGISTRO), "righe")

esegui(False)

print("Terza, senza ripartenza:", conta_righe(REGISTRO), "righe")

print(REGISTRO.read_text(encoding="utf-8"), end="")

# TRABOCCHETTO: senza ripartenza il registro cresce a ogni prova, e i numeri
# progressivi ricominciano da 001 a meta' file. Nessun errore: il sintomo
# e' un 001 che compare due volte.

# Prima esecuzione: 3 righe
# Seconda esecuzione: 3 righe
# Terza, senza ripartenza: 6 righe
# 001 | INFO | avvio: lettura di ordini.csv
# 002 | AVVISO | 3 righe scartate
# 003 | INFO | fine: report scritto
# 001 | INFO | avvio: lettura di ordini.csv
# 002 | AVVISO | 3 righe scartate
# 003 | INFO | fine: report scritto
```

11.4 Il file degli scarti: il motivo di 9.4, di nuovo

```
from pathlib import Path

PERCORSO = Path("dati") / "spedizioni.txt"

BUONE = Path("spedizioni_buone.txt")
SCARTI = Path("spedizioni_scarti.txt")

CAMPI_ATTESI = 3

ZONE_VALIDE = ["NORD", "CENTRO", "SUD", "ISOLE"]

def e_numero(testo):

    pulito = testo.strip().replace(",", ".")

    return pulito.replace(".", "", 1).isdecimal()

def motivo_scarto(campi):

    # Il motivo di 9.4, oppure "" se il record e' buono.

    if len(campi) != CAMPI_ATTESI:

        return "numero di campi"

    if not e_numero(campi[2]):

        return "peso non numerico"

    if campi[1].strip().upper() not in ZONE_VALIDE:

        return "zona sconosciuta"

    return ""
```

11.4 Due file aperti insieme: buone ripulite, scarti com'erano

```
with open(PERCORSO, "r", encoding="utf-8") as f:

    righe = f.readlines()

# 1. Due file aperti INSIEME. Le tonde attorno ai due open permettono di
#    andare a capo (da Python 3.10). All'uscita si chiudono tutti e due.

with (open(BUONE, "w", encoding="utf-8") as buone,
      open(SCARTI, "w", encoding="utf-8") as scarti):

    for riga in righe:

        riga = riga.rstrip("\n")

        if riga.strip() == "" or riga.startswith("#"):

            continue

        campi = riga.split(";")

        motivo = motivo_scarto(campi)

        if motivo == "":

            # 2. Nel file buono va il record RIPULITO, non la riga grezza.

            peso = float(campi[2].strip().replace(",", "."))

            print(f"{campi[0].strip()};{campi[1].strip().upper()};{peso}",
                  file=buone)

        else:

            # 3. Nello scarto va la riga COM'ERA, perche' chi la corregge
            #    deve riconoscerla, con il motivo accanto.

            print(f"{riga} <- {motivo}", file=scarti)

# TRABOCCHETTO: aprire lo scarto in "w" DENTRO il for lo riapre e lo
#    svuota a ogni riga cattiva: alla fine contiene solo l'ultima, ESTERO.
```

11.4 Rileggere i due file: ventidue più tre

```
with open(BUONE, "r", encoding="utf-8") as f:
    righe_buone = f.readlines()

with open(SCARTI, "r", encoding="utf-8") as f:
    righe_scarti = f.readlines()

print(BUONE.name, "->", len(righe_buone), "righe")
print(SCARTI.name, "->", len(righe_scarti), "righe")
print("Riga 21 riscritta pulita:", righe_buone[16], end="")

for riga in righe_scarti:
    print(" " + riga, end="")

# spedizioni_buone.txt -> 22 righe
# spedizioni_scarti.txt -> 3 righe
# Riga 21 riscritta pulita: LS-2026-MI-0999;NORD;5.0
#   LS-2026-TO-0990;NORD   <- numero di campi
#   LS-2026-BA-1001;SUD;dodici   <- peso non numerico
#   LS-2026-GE-1002;ESTERO;3.0   <- zona sconosciuta
```

11.5 Rileggere quello che si è scritto: scrivi e collauda

```
from pathlib import Path

REPORT = Path("report_colli.txt")

INTESTAZIONE = "codice;zona;peso_kg"

BUONE = ["LS-2026-MI-0042;NORD;12.5", "LS-2026-NA-0913;SUD;6.7",
         "LS-2026-PA-1571;ISOLE;8.5"]

def scrivi_report(percorso, intestazione, righe, a_capo):
    # Scrive intestazione e righe, chiudendo ognuna con a_capo.
    with open(percorso, "w", encoding="utf-8") as f:
        f.write(intestazione + a_capo)
        for riga in righe:
            f.write(riga + a_capo)

def collauda(percorso, attese):
    # Rilegge il file e confronta le righe con quelle attese.
    with open(percorso, "r", encoding="utf-8") as f:
        rilette = len(f.readlines())

    if rilette == attese:
        print(f"[OK] {percorso.name}: {rilette} righe, come previsto")
    else:
        print(f"[ERRORE] {percorso.name}: attese {attese}, rilette {rilette}")
```

11.5 \n dimenticato lo vede solo il collaudo

1. Il conto atteso si fa PRIMA di rileggere: intestazione piu' righe buone.

```
scrivi_report(REPORT, INTESTAZIONE, BUONE, "\n")
```

```
collauda(REPORT, 1 + len(BUONE))
```

2. Lo stesso report con l'a capo dimenticato: e' l'errore di writelines

di \$6.4, e sullo schermo nessuno lo vede.

```
scrivi_report(REPORT, INTESTAZIONE, BUONE, "")
```

```
collauda(REPORT, 1 + len(BUONE))
```

TRABOCCHETTO: rileggere e confrontare con len(BUONE) invece di

1 + len(BUONE) da' [ERRORE] su un file giusto. Il collaudo e' codice

anche lui, e sbaglia come il resto.

[OK] report_colli.txt: 4 righe, come previsto

[ERRORE] report_colli.txt: attese 4, rilette 1

12. JSON: SALVARE LISTE E DIZIONARI COSÌ COME SONO

Nomi dei campi e tipi che sopravvivono alla chiusura del programma

12.1 Perché JSON: i record con i nomi dei campi

```
import json

# Lo stesso ordine di 07.8 in due formati. Nel delimitato i campi non hanno
# nome e sono tutti testo; in JSON hanno un nome e un tipo.

riga_delimitata = "NS-1001;AUDIO;99.80;2"

ordine = {"codice": "NS-1001", "categoria": "AUDIO", "importo": 99.80,
          "pezzi": 2}

# json.dumps restituisce il TESTO che finirebbe nel file (dettagli in 12.4).
testo_json = json.dumps(ordine, indent=2)

print(riga_delimitata)
print(testo_json)

# Il viaggio di ritorno: dal delimitato arriva una str, da JSON un int.
campi = riga_delimitata.split(";")
riletto = json.loads(testo_json)

print(type(campi[3]), campi[3] * 2)
print(type(riletto["pezzi"]), riletto["pezzi"] * 2)

# TRABOCCHETTO: "2" * 2 non da' errore, da' "22": la moltiplicazione di
# stringhe del Giorno 02. Dal delimitato si converte a mano, SEMPRE.
```

12.1 Graffe, quadre, doppi apici: quasi Python

```
NS-1001;AUDIO;99.80;2
{
  "codice": "NS-1001",
  "categoria": "AUDIO",
  "importo": 99.8,
  "pezzi": 2
}
<class 'str'> 22
<class 'int'> 4
```

Delimitato ";": nomi solo in intestazione, tutto testo, per tabelle piatte e fogli di calcolo. JSON: nomi in ogni record, numeri, testo, liste e dizionari, per strutture da salvare e ricaricare. Devo rileggere una struttura uguale a com'era? JSON.

12.2 json.load(): dal file alla lista di dizionari

```
import json

from pathlib import Path

PERCORSO = Path("dati") / "rubrica.json"

# 1. json.load legge TUTTO il file e ricostruisce la struttura: qui una lista
# di dizionari, uno per contatto. Nessun split, nessuna conversione.

with open(PERCORSO, "r", encoding="utf-8") as f:

    rubrica = json.load(f)

print(type(rubrica), len(rubrica), "contatti")

print(type(rubrica[0]))

# 2. Ogni elemento e' un dizionario del Giorno 07: si legge per chiave.

for contatto in rubrica:

    print(f"{contatto['cognome']:<9}{contatto['nome']:<8}"

          f"{contatto['telefono']:<12}{contatto['reparto']}")

# TRABOCCHETTO: le chiavi sono quelle scritte nel FILE, maiuscole comprese.

# contatto["Cognome"] si ferma con KeyError: 'Cognome'.

# <class 'list'> 5 contatti

# <class 'dict'>

# Serra    Marta    0891234567  Cardiologia

# De Luca  Anna     0829876543  Radiologia

# Ferraro  Paolo     3471122334  Laboratorio analisi

# Nocera   Nicolò    0895566778  Accettazione

# Guarino  Elena     3289900112  Cardiologia
```

12.2 json.dump() con indent=2 e ensure_ascii=False

```
import json

from pathlib import Path

CONTATTO = {"nome": "Nicolò", "reparto": "Accettazione"}

# 1. Con i default: una riga sola, e la o accentata in forma di escape.
with open("predefinito.json", "w", encoding="utf-8") as f:

    json.dump(CONTATTO, f)

# 2. Con i due argomenti nominati: rientri e lettere leggibili.
with open("leggibile.json", "w", encoding="utf-8") as f:

    json.dump(CONTATTO, f, indent=2, ensure_ascii=False)

predefinito = Path("predefinito.json").read_text(encoding="utf-8")
leggibile = Path("leggibile.json").read_text(encoding="utf-8")

# 3. La o accentata VERA c'e' solo nel secondo file.
print("ò nel file predefinito:", "ò" in predefinito)
print("ò nel file leggibile: ", "ò" in leggibile)
print(leggibile)

# TRABOCCHETTO: ensure_ascii=False scrive la o accentata VERA, quindi
#   encoding="utf-8" nell'open diventa indispensabile: senza, su Windows il
#   file esce in un'altra codifica e json.load, rileggendolo in utf-8, si
#   ferma con UnicodeDecodeError ($2.4).
```

12.3 Il viaggio: la tupla torna lista, la chiave 1 torna "1"

```
import json

from pathlib import Path

VIAGGIO = Path("viaggio.json")

# Una tupla dentro una lista (la riga del Giorno 07 $14.1) e una chiave int.

originale = {"righe": [("Rossi Anna", 78)], 1: "uno"}

with open(VIAGGIO, "w", encoding="utf-8") as f:

    json.dump(originale, f)

with open(VIAGGIO, "r", encoding="utf-8") as f:

    riletto = json.load(f)

print(originale)

print(riletto)

print("Uguali:", originale == riletto)

print(type(originale["righe"][0]), "->", type(riletto["righe"][0]))

print("Chiave 1:", 1 in riletto, "- chiave '1':", "1" in riletto)


# TRABOCCHETTO: riletto[1] si ferma con KeyError: 1, anche se stampando il
# dizionario la chiave "sembra" esserci. Gli apici attorno a '1' sono
# l'unico indizio: guardateli.

# {'righe': [('Rossi Anna', 78)], 1: 'uno'}

# {'righe': [['Rossi Anna', 78]], '1': 'uno'}

# Uguali: False

# <class 'tuple'> -> <class 'list'>

# Chiave 1: False - chiave '1': True
```

12.3 CHICCA: json.dumps(("LS", 3)) stampa ["LS", 3]

```
import json

riga = ("LS", 3)

# dumps la trasforma nel testo JSON: quadre, doppi apici.
testo = json.dumps(riga)

print(testo)

# loads la ricostruisce, ma come LISTA: e' l'unica sequenza che JSON conosce.
riletta = json.loads(testo)

print(riletta)

print(riletta == riga)

# tuple() converte, come int() converte un testo in numero (Giorno 02).
print(tuple(riletta) == riga)

# TRABOCCHETTO: il False qui sopra non dice "i campi sono diversi". Dice
#   "i tipi sono diversi", e stampando i due valori non si vede quasi.

# ["LS", 3]
# ['LS', 3]
# False
# True
```

12.4 dumps() e loads(): le stesse operazioni su una stringa

```
import json

RIEPILOGO = {"ordini": 12, "pezzi": 21, "fatturato": 1457.6,
             "categorie": ["AUDIO", "INFORMATICA", "ACCESSORI", "VIDEO"]}

# 1. dumps restituisce una str: si puo' stampare, contare, controllare.
testo = json.dumps(RIEPILOGO, indent=2)
print(type(testo), len(testo.splitlines()))
print(testo)

# 2. loads ricostruisce il dizionario: i numeri tornano numeri.
di_nuovo = json.loads(testo)
print(f"Scontrino medio: {di_nuovo['fatturato'] / di_nuovo['ordini']:.2f}")

# TRABOCCHETTO: json.dumps(RIEPILOGO, f) non scrive nel file: dumps non
# vuole un file, e il secondo argomento posizionale lo rifiuta con
# TypeError. Con il file si usa dump, senza s.
```

12.4 Il riepilogo di 07.8, pronto per un altro programma

```
<class 'str'> 11
{
  "ordini": 12,
  "pezzi": 21,
  "fatturato": 1457.6,
  "categorie": [
    "AUDIO",
    "INFORMATICA",
    "ACCESSORI",
    "VIDEO"
  ]
}
Scontrino medio: 121.47
```

12.5 Object of type function is not JSON serializable

```
import json

def calcola_totale(prezzo, quantita):
    return prezzo * quantita

riepilogo = {"ordini": 12, "calcolo": calcola_totale}
testo = json.dumps(riepilogo)

Traceback (most recent call last):
  File "prova.py", line 9, in <module>
    testo = json.dumps(riepilogo)
  [... righe interne del modulo json ...]
TypeError: Object of type function is not JSON serializable
when serializing dict item 'calcolo'
```

Alla riga 8 manca la chiamata: nel dizionario è finita la FUNZIONE, l'errore E5 del Giorno 06. TRABOCCHETTO: Python 3.14 aggiunge la riga che nomina la chiave colpevole, 'calcolo': leggetela. Questo errore non si previene con un if: si CORREGGE nel codice.

12.6 Il JSON malformato: JSONDecodeError

```
import json
from pathlib import Path

ROTTTO = Path("rubrica_rotta.json")
# Due contatti, e fra il primo e il secondo manca la virgola.
with open(ROTTTO, "w", encoding="utf-8") as f:
    f.write('[\n')
    f.write('  {"cognome": "Serra", "nome": "Marta"}\n')
    f.write('  {"cognome": "De Luca", "nome": "Anna"}\n')
    f.write(']\n')

with open(ROTTTO, "r", encoding="utf-8") as f:
    rubrica = json.load(f)

Traceback (most recent call last):
  File "prova.py", line 13, in <module>
    rubrica = json.load(f)
  [... righe interne del modulo json ...]
json.decoder.JSONDecodeError: Expecting ',' delimiter: line 3 column 3 (char 44)
```

"Mi aspettavo una virgola, alla riga 3, colonna 3, carattere 44": riga e colonna del file JSON, non del programma. Punta SUBITO DOPO l'errore: la virgola mancava in fondo alla riga 2. TRABOCCHETTO: il nome completo è json.decoder.JSONDecodeError; in rete cercate l'ultima parte.

12.6 Il limite di quello che un if può controllare

Per sapere con un if se un file è JSON valido, bisognerebbe fare lo stesso lavoro che fa `json.load()`: leggere tutto, riconoscere graffe, virgolette e virgole.

Non esiste una `.isdecimal()` per il JSON.

Esiste uno strumento che raccoglie l'errore dopo che è successo: ci arriveremo quando arriveremo alle eccezioni.
Il controllo preventivo resta la prima linea di difesa, anche dopo.

Oggi la difesa è di PROCESSO: i file JSON li scrive il programma con `json.dump()`, non una persona a mano.
Un file scritto da `json.dump()` è valido per costruzione.

12.7 Il ciclo carica → modifica → salva: tre funzioni

```
import json

from pathlib import Path

ORIGINALE = Path("dati") / "rubrica.json"
SALVATA = Path("output") / "rubrica.json"

def carica_rubrica(percorso):
    # Il controllo preventivo di §8.2: il file che manca non ferma niente.
    if not percorso.is_file():
        return []

    with open(percorso, "r", encoding="utf-8") as f:
        return json.load(f)

def aggiungi_contatto(rubrica, cognome, nome, telefono, reparto):
    # Accoda un contatto alla rubrica ricevuta (la modifica sul posto).
    rubrica.append({"cognome": cognome, "nome": nome,
                    "telefono": telefono, "reparto": reparto})

def salva_rubrica(rubrica, percorso):
    # Scrive la rubrica in JSON leggibile, creando la cartella se serve.
    percorso.parent.mkdir(parents=True, exist_ok=True)
    with open(percorso, "w", encoding="utf-8") as f:
        json.dump(rubrica, f, indent=2, ensure_ascii=False)
```

12.7 Primo avvio, secondo avvio: un programma che si ricorda

```
# PRIMO AVVIO: si parte dall'originale, si aggiunge, si salva.

rubrica = carica_rubrica(ORIGINALE)

print("Primo avvio, contatti:", len(rubrica))

aggiungi_contatto(rubrica, "Palladino", "Roberto", "0897654321", "Radiologia")

salva_rubrica(rubrica, SALVATA)

print("Salvata in:", SALVATA.as_posix())


# SECONDO AVVIO: in un programma vero la lista di prima non esiste piu'. Qui
# l'assegnazione la sostituisce per intero con quello che c'e' nel FILE.

rubrica = carica_rubrica(SALVATA)

print("Secondo avvio, contatti:", len(rubrica))

ultimo = rubrica[-1]

print("Ultimo:", ultimo["cognome"], ultimo["nome"], ultimo["telefono"])

print("Originale intatto:", len(carica_rubrica(ORIGINALE)))


# TRABOCCHETTO: salvare su ORIGINALE invece che su SALVATA funziona la prima
# volta. Dalla seconda il "dato di partenza" contiene gia' le vostre prove,
# e nessun collaudo ripete piu' lo stesso risultato.


# Primo avvio, contatti: 5

# Salvata in: output/rubrica.json

# Secondo avvio, contatti: 6

# Ultimo: Palladino Roberto 0897654321

# Originale intatto: 5
```

13. CARTELLE ED ELENCHI DI FILE

Che cosa c'è, quanti sono, quanto sono lunghi: sempre con `sorted()`

13.1 iterdir(): l'elenco dei file, sempre con sorted()

```
from pathlib import Path

CARTELLA_DATI = Path("dati")

voci = 0

# 1. sorted() costruisce una lista ordinata, e iterdir() non viene toccato
# (Giorno 07 $8.1). L'ordine e' alfabetico, uguale su ogni computer.
for percorso in sorted(CARTELLA_DATI.iterdir()):

    # 2. I nomi che cominciano con il punto sono file nascosti del sistema
    # (.DS_Store sul Mac): non sono dati, si saltano.
    if percorso.name.startswith("."):
        continue

    voci += 1

    # 3. Si stampa .name ($7.5); la barra finale segnala le sottocartelle.
    if percorso.is_dir():
        print(" " + percorso.name + "/")
    else:
        print(" " + percorso.name)

print("Voci:", voci)

# TRABOCCHETTO: senza sorted() su questa macchina l'elenco puo' anche uscire
# in ordine, e su un'altra no. Il difetto non si vede dove lo scrivete: si
# vede dove lo consegnate.
```

13.1 Sedici voci in dati/, in ordine alfabetico

```
anagrafiche.txt
archivio/
candidature.txt
codici_colli.txt
configurazione_filtro.txt
contatti_nord.txt
contatti_sud.txt
isee_richieste.txt
lettera_candidatura.txt
listino_zone.txt
messaggi.txt
ordini.csv
rubrica.json
spedizioni.txt
ticket.txt
vuoto.txt
```

Voci: 16

archivio/ è l'unica sottocartella: la barra finale la distingue dai file. È la stessa cartella che al §8.1 esisteva senza essere un file.

13.1 glob("*.csv"): l'elenco filtrato sul nome

```
from pathlib import Path

CARTELLA_DATI = Path("dati")

for modello in ["*.csv", "*.json", "*.xlsx"]:

    # glob restituisce le voci che corrispondono al modello; sorted() le
    # ordina e le mette in una lista, che si puo' contare con len().

    trovati = sorted(CARTELLA_DATI.glob(modello))

    print(f"{modello:<8}{len(trovati)} file")

    for percorso in trovati:

        print("    ", percorso.name)

# TRABOCCHETTO: nessun file .xlsx non e' un errore: glob restituisce una
# lista vuota e il for fa zero giri (Giorno 05 $7.3). Se "zero file" e' un
# problema, lo dovete controllare voi con un if.

# *.csv    1 file
#         ordini.csv
# *.json   1 file
#         rubrica.json
# *.xlsx   0 file
```

13.2 Filtrare con .suffix e contare le righe di ogni file

```
from pathlib import Path

CARTELLA_DATI = Path("dati")

def conta_righe(percorso):
    # Il contatore del Giorno 05 su un for che scorre il file ($4.3).
    quante = 0
    with open(percorso, "r", encoding="utf-8") as f:
        for riga in f:
            quante += 1
    return quante

print(f"{'FILE':<27}{'RIGHE':>5}")

for percorso in sorted(CARTELLA_DATI.iterdir()):
    if percorso.name.startswith(".") or not percorso.is_file():
        continue
    # .suffix e' l'estensione CON il punto: ".txt", non "txt".
    if percorso.suffix != ".txt":
        continue
    print(f"{percorso.name:<27}{conta_righe(percorso):>5}")

# TRABOCCHETTO: confrontare .suffix con "txt", senza punto, da' sempre
# falso: la tabella esce con la sola intestazione, e nessun errore.
```

13.2 Righe, mai byte

FILE	RIGHE
anagrafiche.txt	12
candidature.txt	12
codici_colli.txt	14
configurazione_filtro.txt	13
contatti_nord.txt	6
contatti_sud.txt	6
isee_richieste.txt	12
lettera_candidatura.txt	12
listino_zone.txt	5
messaggi.txt	15
spedizioni.txt	27
ticket.txt	30
vuoto.txt	0

percorso.stat().st_size darebbe i byte, ma su Windows ogni fine riga ne occupa due e lo stesso file "pesa" di più: le righe sono le stesse ovunque. vuoto.txt esiste, è un file e ha zero righe: il controllo del §8.1 lo lascia passare, ed è giusto che il conteggio lo dica.

13.3 shutil.copy(): la copia prima di sovrascrivere

```
import shutil

from pathlib import Path

ORIGINALE = Path("dati") / "ticket.txt"
CARTELLA_BACKUP = Path("output") / "backup"

def conta_righe(percorso):
    with open(percorso, "r", encoding="utf-8") as f:
        return len(f.readlines())

# 1. La cartella di destinazione PRIMA della copia (§8.3).
CARTELLA_BACKUP.mkdir(parents=True, exist_ok=True)

# 2. Stesso nome, altra cartella: .name riusa il nome dell'originale.
copia = CARTELLA_BACKUP / ORIGINALE.name
shutil.copy(ORIGINALE, copia)

print("Copia creata:", copia.as_posix())
print("Righe originale:", conta_righe(ORIGINALE))
print("Righe copia:", conta_righe(copia))

# 3. Controprova piu' forte del conteggio: il testo intero e' identico.
uguali = ORIGINALE.read_text(encoding="utf-8") == copia.read_text(encoding="utf-8")
print("Testo identico:", uguali)

# TRABOCCHETTO: senza il mkdir, shutil.copy verso una cartella che non c'e'
# si ferma con FileNotFoundError, come l'open di §8.3.
```

13.4 Il riepilogo di una cartella: quanti file, quante righe

```
from pathlib import Path

CARTELLA_DATI = Path("dati")

# conta_righe() e' quella del 13.3.


file_contati = 0
righe_totali = 0
per_estensione = {}

for percorso in sorted(CARTELLA_DATI.iterdir()):

    if percorso.name.startswith(".") or not percorso.is_file():

        continue

    file_contati += 1

    # L'ACCUMULATORE somma le righe; il CONTA-OCCORRENZE conta le estensioni.

    righe_totali += conta_righe(percorso)

    estensione = percorso.suffix

    per_estensione[estensione] = per_estensione.get(estensione, 0) + 1


print("File in dati/:", file_contati)

print("Righe in tutto:", righe_totali)

for estensione, quanti in per_estensione.items():

    print(f"  {estensione:<6}{quanti:>3}")


# TRABOCCHETTO: senza "not percorso.is_file()" il ciclo prova ad aprire
#  archivio/ come un file, e si ferma con IsADirectoryError sul Mac e su
#  Linux, PermissionError su Windows: lo stesso difetto, due nomi.
```

14. LO SCHEMA COMPLETO: LEGGERE, VALIDARE, ELABORARE, SCRIVERE

Niente di nuovo: tutto quello che avete, messo in ordine

14.1 I quattro passi: leggere, validare, elaborare, scrivere

Passo	Che cosa fa	Da quale giornata
Leggere	apre il file, scorre le righe, toglie l'a capo	oggi, cap. 3-5
Validare	salta vuote e commenti, conta i campi, controlla numeri e codici	Giorno 04 (if, .isdecimal()), Giorno 05 (continue, contatori), oggi cap. 9
Elaborare	accumula, conta per chiave, ordina, fa i conti	Giorno 05 (accumulatore), Giorno 06 (funzioni), Giorno 07 (liste, dizionari)
Scrivere	compone le righe, le scrive, rilegge per controllo	Giorno 03 (incolonnamento), oggi cap. 6 e 11

Ogni programma che lavora su file ha la stessa forma, in quattro passi. Due su quattro sono nuovi, leggere e scrivere. Gli altri due sono il grosso del programma, e li avete già scritti.

14.1 Lo schema nella sua forma più piccola

```
from pathlib import Path

ENTRATA = Path("pesi.txt")
USCITA = Path("totale.txt")

with open(ENTRATA, "w", encoding="utf-8") as f:
    f.write("12.5\n7,3\ndodici\n")

# 1. LEGGERE: le righe dal file.
with open(ENTRATA, "r", encoding="utf-8") as f:
    righe = f.readlines()

# 2. VALIDARE: il controllo di 9.3, riga per riga; le buone in una lista.
pesi = []

for riga in righe:
    testo = riga.strip().replace(",", ".")
    if testo.replace(".", "", 1).isdecimal():
        pesi.append(float(testo))

# 3. ELABORARE: un conto del Giorno 07 su una lista pulita.
totale = sum(pesi)

# 4. SCRIVERE: il risultato su file, con la print di §6.3.
with open(USCITA, "w", encoding="utf-8") as f:
    print(f"Colli validi: {len(pesi)} su {len(righe)}", file=f)
    print(f"Peso totale: {totale:.1f} kg", file=f)

# TRABOCCHETTO: mescolare i passi, cioè' sommare mentre si valida e si
#   scrive, funziona su tre righe. Su trenta, il giorno che il report e'
#   sbagliato, non sapete in quale dei quattro passi cercare.

print(USCITA.read_text(encoding="utf-8"), end="")
```

14.2 Lo schema per intero: costanti e LEGGERE

```
from pathlib import Path

CARTELLA_DATI = Path("dati")
CARTELLA_OUTPUT = Path("output")
FILE_SPEDIZIONI = CARTELLA_DATI / "spedizioni.txt"
FILE_LISTINO = CARTELLA_DATI / "listino_zone.txt"
FILE_REPORT = CARTELLA_OUTPUT / "report_zone.txt"
CAMPI_ATTESI = 3

def righe_utili(percorso):
    # LEGGERE: la pulizia standard di $5.2-5.3, scritta una volta sola.
    utili = []
    with open(percorso, "r", encoding="utf-8") as f:
        for riga in f:
            riga = riga.rstrip("\n")
            if riga.strip() == "" or riga.startswith("#"):
                continue
            utili.append(riga)
    return utili

def e_numero(testo):
    pulito = testo.strip().replace(",", ".")
    return pulito.replace(".", "", 1).isdecimal()
```

14.2 VALIDARE: carica_listino() e carica_spedizioni()

```
def carica_listino(percorso):  
    listino = {}  
  
    for riga in righe_utili(percorso):  
        zona, prezzo = riga.split(";")  
  
        listino[zona.strip().upper()] = float(prezzo)  
  
    return listino  
  
def carica_spedizioni(percorso, listino):  
    # VALIDARE: lista di tuple (codice, zona, peso) buone e numero di scarti.  
  
    spedizioni = []  
  
    scartate = 0  
  
    for riga in righe_utili(percorso):  
        campi = riga.split(";")  
  
        if len(campi) != CAMPI_ATTESI or not e_numero(campi[2]):  
            scartate += 1  
  
            continue  
  
        zona = campi[1].strip().upper()  
  
        # Le zone valide sono le chiavi del listino: un elenco in meno da  
  
        # tenere allineato a mano.  
  
        if zona not in listino:  
            scartate += 1  
  
            continue  
  
        peso = float(campi[2].strip().replace(",", "."))  
  
        spedizioni.append((campi[0].strip(), zona, peso))  
  
    return spedizioni, scartate
```

14.2 ELABORARE e SCRIVERE: tre dizionari, una lista di righe

```
def costi_per_zona(spedizioni, listino):  
    # ELABORARE: tre dizionari per zona, cioe' colli, chili e costo.  
  
    colli = {}  
  
    chili = {}  
  
    costi = {}  
  
    for codice, zona, peso in spedizioni:  
        # Tre volte l'aggregazione del Giorno 07 $12.4.  
  
        colli[zona] = colli.get(zona, 0) + 1  
  
        chili[zona] = chili.get(zona, 0) + peso  
  
        costi[zona] = costi.get(zona, 0) + listino[zona]  
  
    return colli, chili, costi  
  
def componi_report(spedizioni, scartate, colli, chili, costi):  
    # SCRIVERE, prima meta': le righe del report, senza stamparle (11.1).  
  
    righe = ["LOGISUD - Colli del giorno per zona",  
            f"Righe buone: {len(spedizioni)}   scartate: {scartate}",  
            f"{'ZONA':<8}{ 'COLLI':>6}{ 'KG':>8}{ 'COSTO':>8}"]  
  
    for zona in colli:  
        righe.append(f"{'zona':<8}{colli[zona]:>6}{chili[zona]:>8.1f}"  
                    f"{'costi[zona]:>8.2f}")  
  
    righe.append(f"{'TOTALE':<8}{sum(colli.values()):>6}"  
                f"{'sum(chili.values()):>8.1f}{sum(costi.values()):>8.2f}")  
  
    return righe
```

14.2 main(), il regista: i quattro passi in ordine

```
def scrivi_report(righe, percorso):  
    # SCRIVERE, seconda meta': le righe su file, cartella compresa.  
  
    percorso.parent.mkdir(parents=True, exist_ok=True)  
  
    with open(percorso, "w", encoding="utf-8") as f:  
        for riga in righe:  
            print(riga, file=f)  
  
def main():  
    # Il regista: chiama i quattro passi in ordine, e non fa conti suoi.  
  
    listino = carica_listino(FILE_LISTINO)  
  
    spedizioni, scartate = carica_spedizioni(FILE_SPEDIZIONI, listino)  
  
    colli, chili, costi = costi_per_zona(spedizioni, listino)  
  
    righe = componi_report(spedizioni, scartate, colli, chili, costi)  
  
    for riga in righe:  
        print(riga)  
  
    scrivi_report(righe, FILE_REPORT)  
  
    # Il collaudo di 11.5: il file riletto ha le righe che doveva avere.  
  
    scritte = len(righe_utili(FILE_REPORT))  
  
    print(f"[OK] {FILE_REPORT.as_posix()}: {scritte} righe")  
  
# TRABOCCHETTO: il costo si somma con listino[zona], accesso DIRETTO e non  
# .get(). Qui e' giusto: carica_spedizioni ha gia' scartato le zone fuori  
# listino, e se una ne passasse lo stesso il KeyError direbbe che il  
# filtro di prima ha un buco. Un .get(zona, 0) lo nasconderebbe.  
  
if __name__ == "__main__":  
    main()
```

14.2 I numeri del capitolo 9: 22 buone, 3 scartate, 214 euro

```
LOGISUD - Colli del giorno per zona
Righe buone: 22   scartate: 3
ZONA      COLLI      KG      COSTO
NORD      5        56.1    42.50
CENTRO    5        90.7    35.00
SUD       7       141.1    66.50
ISOLE     5        78.6    70.00
TOTALE    22       366.5   214.00
[OK] output/report_zone.txt: 8 righe
```

Leggete main() da sola, senza aprire le funzioni: una manciata di righe che raccontano il programma. È la leggibilità per livelli del Giorno 06 §7.2.

14.3 Dove si fermerebbe il programma: un if non basta sempre

Dove	Che cosa succede	Perché un if non basta
carica_listino()	il file è stato cancellato un istante dopo il controllo	fra is_file() e open() il mondo cambia (§8.5)
qualunque open()	il file è aperto da un altro programma, o mancano i permessi	nessun metodo di Path lo dice in anticipo
carica_listino()	un prezzo scritto 8,50 o abc	il listino non passa per e_numero(): ValueError
e_numero()	un peso scritto -3.5 o 1e3	il controllo rifiuta numeri che Python accetterebbe
un JSON nel programma	il file è malformato	l'unico controllo possibile è leggerlo (§12.6)

Il programma del 14.2 regge tutti i difetti DENTRO le righe del file. Questi cinque punti no. Quando lo strumento che raccoglie l'errore arriverà, si partirà da questo programma e da questa tabella: le righe da proteggere sono già tutte qui.

14.3 Il controllo e float() non sono d'accordo

```
def e_numero(testo):  
    # Vero se il testo, ripulito, e' un numero decimale senza segno.  
    pulito = testo.strip().replace(",", ".", "")  
    return pulito.replace(".", "", 1).isdecimal()  
  
for testo in ["-3.5", "1e3"]:  
    # Il controllo dice no; float() converte senza problemi.  
    print(f"{testo:<10}controllo: {e_numero(testo)}    float(): {float(testo)}")  
  
# TRABOCCHETTO: rendere e_numero piu' furba, con un if per il segno e uno  
#   per la "e", e' una rincorsa senza fine: le regole di float() sono molte  
#   di piu'. Il controllo preventivo ha un confine, e questo e' il suo.  
  
# -3.5      controllo: False    float(): -3.5  
# 1e3       controllo: False    float(): 1000.0
```

15. ERRORI TIPICI DELLA GIORNATA

Cinque danno un traceback. Quattro non danno niente: E2, E3, E5, E8

E1 — FileNotFoundError: la directory di lavoro sbagliata

```
from pathlib import Path
```

```
percorso = Path("spedizioni.txt")  
with open(percorso, "r", encoding="utf-8") as f:  
    righe = f.readlines()
```

```
Traceback (most recent call last):
```

```
  File "prova.py", line 4, in <module>
```

```
    with open(percorso, "r", encoding="utf-8") as f:
```

```
FileNotFoundError: [Errno 2] No such file or directory: 'spedizioni.txt'
```

Il file esiste, ma in dati/: il relativo si risolve dalla directory di lavoro (§7.1). TRABOCCHETTO: l'ultima riga ripete il percorso ESATTO cercato, leggetelo ad alta voce.

Cure: Path("dati") / nome; negli esercizi il percorso ancorato del §7.4; is_file() del §8.1.

E2 — Il confronto che non scatta mai: il \n attaccato alla riga

```
from pathlib import Path

NOMI = Path("nomi.txt")

with open(NOMI, "w", encoding="utf-8") as f:
    f.write("Serra\nRinaldi\nSerra\n")

senza_pulizia = 0
con_pulizia = 0

with open(NOMI, "r", encoding="utf-8") as f:
    for riga in f:
        # La riga letta vale "Serra\n": sei caratteri, non cinque.
        if riga == "Serra":
            senza_pulizia += 1
        if riga.rstrip("\n") == "Serra":
            con_pulizia += 1

# TRABOCCHETTO: nessun errore, solo un contatore fermo a zero. E' il
# sintomo del Giorno 05 $14.5 con una causa nuova: un carattere invisibile.
print("Senza pulizia:", senza_pulizia)
print("Con rstrip:", con_pulizia)

# Senza pulizia: 0
# Con rstrip: 2
```

E3 — "w" al posto di "a": il registro svuotato in silenzio

```
from pathlib import Path

REGISTRO = Path("accessi.txt")

# Tre "serate" di accessi allo sportello: sei nomi in tutto.
SERATE = [{"Serra", "Rinaldi"}, {"De Luca"}, {"Motta", "Nocera", "Bellini"}]

def scrivi_serate(modalita):
    with open(REGISTRO, "w", encoding="utf-8") as f:
        f.write("")

    for accessi in SERATE:
        # Ogni serata riapre il file: e' QUI che la modalita' decide.
        with open(REGISTRO, modalita, encoding="utf-8") as f:
            for nome in accessi:
                f.write(nome + "\n")

    with open(REGISTRO, "r", encoding="utf-8") as f:
        return len(f.readlines())

# TRABOCCHETTO: con "w" ogni apertura cancella la serata prima. Resta solo
# l'ultima, e il file ha un aspetto perfettamente normale.
print('Con "w":', scrivi_serate("w"), "righe su 6")
print('Con "a":', scrivi_serate("a"), "righe su 6")

# Con "w": 3 righe su 6
# Con "a": 6 righe su 6
```

E4 — TypeError: write() argument must be str, not int

```
with open("conteggio.txt", "w", encoding="utf-8") as f:  
    f.write(42)
```

```
Traceback (most recent call last):  
  File "prova.py", line 2, in <module>  
    f.write(42)  
TypeError: write() argument must be str, not int
```

write() accetta solo stringhe. È un errore del programmatore, non dei dati: si corregge, non si previene. TRABOCCHETTO: la cura non è str(42) sparso ovunque, è la f-string che compone la riga intera (§6.2). In alternativa print(42, file=f) converte da sola (§6.3).

E5 — Il secondo giro sul file già letto: zero righe, zero errori

```
from pathlib import Path

PROVA = Path("tre_righe.txt")

with open(PROVA, "w", encoding="utf-8") as f:
    f.write("NORD\nSUD\nISOLE\n")

with open(PROVA, "r", encoding="utf-8") as f:
    primo = 0
    for riga in f:
        primo += 1

    # Il segno di lettura e' in fondo al file: il secondo for parte da li'.
    secondo = 0
    for riga in f:
        secondo += 1

# TRABOCCHETTO: il secondo ciclo non da' errore, fa zero giri (Giorno 05
# $7.3). Un file aperto si legge una volta; una lista, quante volte volete.
print("Primo giro:", primo, "righe")
print("Secondo giro:", secondo, "righe")

# Primo giro: 3 righe
# Secondo giro: 0 righe
```

E6 — ValueError nell'unpacking: la riga con un campo in meno

```
riga = "Serra Ida;4;81"
nome, anni, punteggio, esito = riga.split(";")

Traceback (most recent call last):
  File "prova.py", line 2, in <module>
    nome, anni, punteggio, esito = riga.split(";")
ValueError: not enough values to unpack (expected 4, got 3)
```

È l'ultima riga di candidature.txt: manca l'esito. Quattro nomi, tre pezzi. TRABOCCHETTO: "not enough" è identico su tutte le versioni; "too many" su Python 3.14 aggiunge il numero di pezzi arrivati. La cura è contare prima: len(campi) != CAMPI_ATTESI (§9.2).

E7 — UnicodeDecodeError e lettere storpiate: la codifica

```
from pathlib import Path

PROVA = Path("lettera.txt")
# Scritto con la codifica di Windows italiano, riletto in UTF-8.
with open(PROVA, "w", encoding="cp1252") as f:
    f.write("perché\n")
with open(PROVA, "r", encoding="utf-8") as f:
    testo = f.read()
```

```
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe9 in position 5: invalid continuation byte
```

position 5 è la sesta lettera, la é. TRABOCCHETTO: il caso inverso non dà errore: UTF-8 riletto come latin-1 stampa perchÃ© e va avanti (§2.4), e il danno silenzioso è il peggiore. La cura: encoding="utf-8" in OGNI open(), in scrittura e in lettura (§2.3).

E8 — Il JSON riletto è diverso: tuple e chiavi numeriche

```
import json

LISTINO = {1: ("NORD", 8.5)}

# Andata e ritorno in una riga: dumps verso il testo, loads indietro.
riletto = json.loads(json.dumps(LISTINO))

# TRABOCCHETTO: nessun errore. La chiave 1 e' diventata "1" e la tupla una
# lista, e il primo riletto[1] del programma si fermerebbe con KeyError: 1.
print(riletto)

print(riletto == LISTINO)

# {'1': ['NORD', 8.5]}

# False
```

Si previene prevedendo: chiavi stringa e liste, se la struttura deve viaggiare in JSON; tuple() o int() dopo il load, se serve il tipo originale (§12.3).

E9 — FileNotFoundError in scrittura: la cartella che manca

```
from pathlib import Path

percorso = Path("output") / "report.txt"
with open(percorso, "w", encoding="utf-8") as f:
    f.write("prova\n")

Traceback (most recent call last):
  File "prova.py", line 4, in <module>
    with open(percorso, "w", encoding="utf-8") as f:
FileNotFoundError: [Errno 2] No such file or directory: 'output/report.txt'
```

Il file doveva essere CREATO, eppure "non esiste": manca la cartella, e "w" crea il file, non le cartelle sopra. TRABOCCHETTO: su Windows il percorso nel messaggio ha la barra rovesciata, ed è lo stesso errore. La cura: percorso.parent.mkdir(parents=True, exist_ok=True) prima dell'open (§8.3).

RIEPILOGO DEL GIORNO

Ventuno righe da riguardare stasera

Riepilogo del giorno (1/2)

La variabile muore alla chiusura del programma, il file resta.
encoding="utf-8" in OGNI open(), in lettura e in scrittura.
with open(percorso, "r", encoding="utf-8") as f: si chiude da solo.
r legge, w cancella e scrive, a accoda, x crea solo se non esiste.
for riga in f legge qualunque dimensione; read(), readlines(), readline()
per i casi particolari. Un file aperto si legge una volta sola.
riga = riga.rstrip("\n"), poi continue su righe vuote e commenti.
write() non va a capo e vuole str; print(..., file=f) va a capo e converte.
Path("dati") / nome; negli esercizi il percorso ancorato a __file__.
is_file() prima di aprire, mkdir(parents=True, exist_ok=True) prima di scrivere.
campi = riga.split(";"), e len(campi) PRIMA dell'unpacking.
e_numero() prima di float(), con i limiti dichiarati: -3.5 e 1e3.
motivo_scarto() restituisce il primo motivo, o "" se la riga è buona.

Riepilogo del giorno (2/2)

L'intestazione si consuma con `f.readline()` prima del `for`.

`carica_...()` restituisce la lista di tuple e gli scarti:

il resto del programma non cambia.

Prima si compone una lista di righe, poi si decide dove scriverla:

`destinazione=None` va a video, un file aperto va sul disco.

Registro: modalità a, numero progressivo al posto dell'orario, ripartenza pulita.

`json.dump(dati, f, indent=2, ensure_ascii=False)` e `json.load(f)`:

le tuple tornano liste, le chiavi tornano stringhe.

`sorted(cartella.iterdir())`, `glob("*.csv")`, `.suffix`, `shutil.copy()`: righe, mai byte.

Leggere, validare, elaborare, scrivere: nuovi sono solo il primo e l'ultimo.

Il confine: oggi si previene con un `if`. Lo strumento che raccoglie

l'errore dopo che è successo arriva con le eccezioni.

Grazie.

shaping the skills of tomorrow

